TRƯỜNG ĐẠI HỌC SÀI GÒN

KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO

Đề Tài: MicroShop – Website thương mại điện tử với Microservices.

Nhóm sinh viên thực hiện – Nhóm 3

Nguyễn Đức Tài - 3121411186

Lâm Kiến Minh - 3121411132

Mai Hải Hậu – 3121411067

Giảng viên hướng dẫn: TS - Đỗ Như Tài

Thành phố Hồ Chí Minh, tháng 05 năm 2025

MỤC LỤC

I, Tổng quan đề tài 1
1, Bối cảnh và vấn đề 1
2, Mục tiêu nghiên cứu 1
3, Phương pháp thực hiện 1
4, Quy trình phát triển phần mềm theo mô hình Scrum. 2
II, Cơ sở lý thuyết 3
1, Lý luận tổng quan kiến trúc phần mềm 3
2, Công nghệ sử dụng 4
3, Kiến trúc Node.js (Back-End và Front-End) 5
4, Frontend 8
5, Backend 10
6, API Gateway 13
7, Data Storage 13
III, Phân tích hệ thống 16
1, Business Context 16
2, Use Case Diagram 20
3, User Story 24
4, Lược đồ dữ liệu mức khái niệm 27
IV, Thiết kế hệ thống 28
1, C4 Model 28
2, Giao tiếp trong ShopMicro 32
3, Khung nhìn mức dữ liệu 34
4, Triển khai hệ thống 37
5, CI/CD với Jenkins 40
6, Resilience: Giám sát hệ thống 45
V, Kết quả đạt được 48

HÌNH ẢNH

Hình 1.3: Sơ đồ hệ thống ShopMicro

Hình 2.2: Sơ đồ hệ thống ShopMicro (Công nghệ sử dụng)

Hình 2.3.1: Kiến trúc N-layer (Backend)

Hình 2.4.1: Công nghệ - Frontend

Hình 2.4.2: Trang chủ ShopMicro

Hình 2.5.1: Công nghệ - Backend

Hình 2.5.2: Message Broker - RabbitMQ

Hình 2.5.3: Caching Data - Redis

Hình 2.6: Công nghệ - API Gateway

Hình 2.7.1: Lược đồ Logic - MySQL

Hình 2.7.2: Lược đồ logic - PostgreSQL

Hình 2.7.3(1): Lược đồ logic - MongoDB

Hình 2.7.3(2): Data Access – MongoDB

Hình 3.2: Use Case Diagram - ShopMicro

Hình 3.2.1 – UC01: Dịch vụ xác thực

Hình 3.2.2 – UC02: Dịch vụ người dùng

Hình 3.2.3 – UC03: Dịch vụ sản phẩm

Hình 3.2.4 – UC04: Dịch vụ thông báo

Hình 3.2.5 – UC05: Dịch vụ đơn hàng

Hình 3.2.6 – UC06: Dịch vụ trò chuyện

Hình 3.2.7 – UC07: Dịch vụ đánh giá

Hình 3.4(1): ERD ShopMicro

Hình 3.4(2): Activity Diagram – Thanh toán

Hình 4.1.1 – System Context Diagram

Hình 4.1.2 – Container Diagram (Ex: Chức năng đăng nhập)

Hình 4.1.3 – Component Diagram (Ex: Chức năng đăng nhập)

Hình 4.2.1: Giao tiếp giữa Client & Services

Hình 4.3.1: MySQL - ORM

Hình 4.3.2: MongoDB – ODM

Hình 4.3.3: PostgreSQL – Raw Query

Hình 4.4(1): Quy trình triển khai hệ thống

Hình 4.4(2): Minh họa Dockerfile

Hình 4.4(3): Minh họa docker-compose.yaml

Hình 4.5.1: Luồng hoạt động của CI/CD

4.5.2(1): Minh họa cách build Jenkin

4.5.2(2): Minh họa các hàm hỗ trợ Pipeline

4.5.2(3): Minh họa thông báo cho Pipeline

4.5.2(4): Minh họa Webhook Github

4.5.2(5): Minh họa build Pipeline thành công

Hình 4.6.2: Giao diện Kibana (Elasticsearch)

Hình 4.6.3: APM Server

Hình 4.6.4: Metricbeat

Hình 4.6.5: Heartbeat

PHÂN CÔNG

Sinh viên thực hiện	Công việc	Thái độ
Mai Hải Hậu	Thực hiện Chương 1, 2, 5: - **Chương 1**: Mục 1, 2 - **Chương 2**: Mục 1, 2, 3 - **Chương 5:** Mục 5 - 8	Tốt
Lâm Kiến Minh	Thực hiện Chương 1, 3 ,5: - **Chương 1**: Mục 3, 4 - **Chương 3**: Mục 1, 2 - **Chương 5:** Mục 1 - 4	Tốt
Nguyễn Đức Tài	Thực hiện Chương 2, 3, 4: - **Chương 2**: Mục 4 - 7 - **Chương 3**: Mục 3, 4 - **Chương 4:** Mục 1 - 6	Tốt
Link project nhóm: https://github.com/td310/ShopMicro		

I, Tổng quan đề tài

1, Bối cảnh và vấn đề

Trong thời đại số hóa hiện nay, thị trường làm việc tự do (freelance) ngày càng phát triển mạnh mẽ. Các nền tảng như Fiverr, Upwork đã góp phần quan trọng vào việc kết nối người thuê và người làm tự do trên toàn cầu. Tuy nhiên, bên cạnh những thành công đó, các nền tảng hiện tại vẫn tồn tại những hạn chế lớn:

- Kiến trúc đơn khối (Monolithic) khiến hệ thống khó bảo trì và mở rộng.
- Trải nghiệm người dùng chưa tối ưu, chưa thân thiện với các thiết bị hiện đại.
- Khả năng cá nhân hóa và tích hợp công nghệ mới còn hạn chế.

Đứng trước các vấn đề trên, hệ thống ShopMicro được đề xuất như một giải pháp nhằm khắc phục những điểm yếu của các nền tảng truyền thống thông qua việc:

- Áp dụng kiến trúc Microservices giúp phân tách chức năng rõ ràng, dễ mở rộng và bảo trì.
- Xây dựng hệ thống SPA (Single Page Application) nhằm nâng cao trải nghiệm người dùng.
- Thiết kế linh hoạt, phù hợp cho các dự án học tập lẫn triển khai thực tế.

2, Mục tiêu nghiên cứu

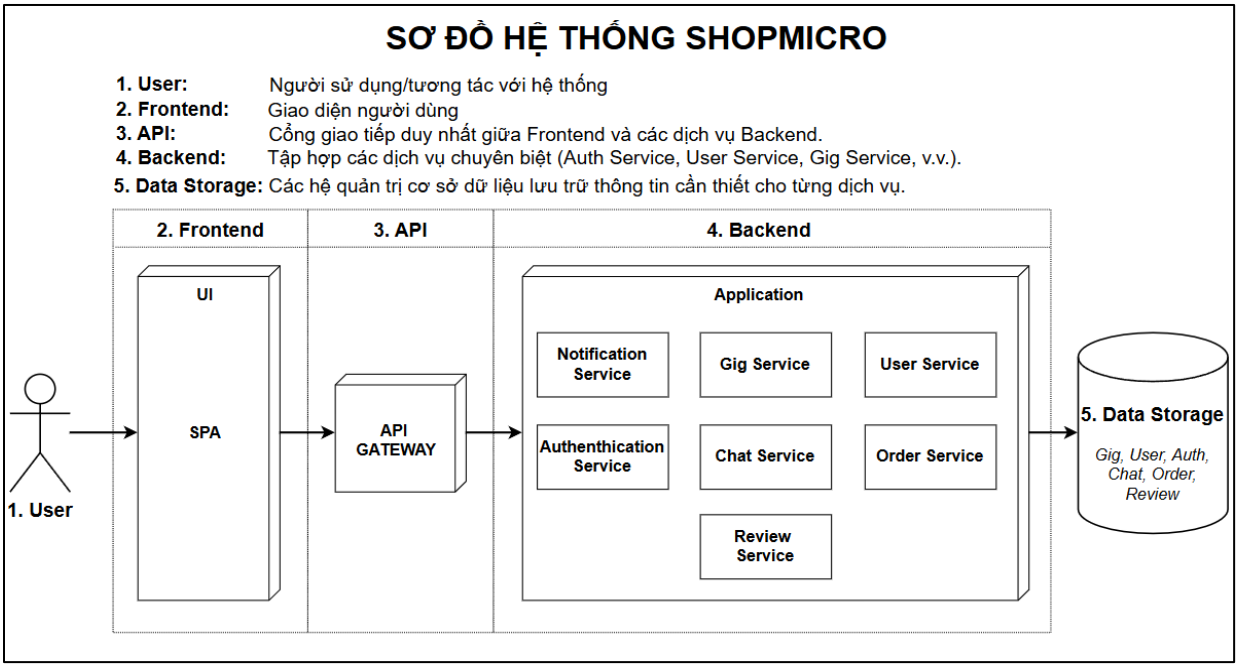
- Xây dựng một nền tảng freelance hiện đại dựa trên kiến trúc microservices.
- Tối ưu hóa trải nghiệm người dùng qua mô hình SPA với ReactJS.
- Phân tích, thiết kế và triển khai hệ thống theo hướng dễ mở rộng, bảo trì và cá nhân hóa.

3, Phương pháp thực hiện

Hệ thống được xây dựng dựa trên mô hình kiến trúc **Microservices**, trong đó toàn bộ hệ thống được chia thành bốn phần chính:

- **Frontend**: Giao diện người dùng được xây dựng riêng biệt, đóng vai trò tương tác trực tiếp với người dùng cuối.
- **API Gateway**: Là cửa ngõ trung gian duy nhất giữa frontend và các dịch vụ backend, giúp phân phối request đến đúng service và thực hiện các tác vụ xác thực, logging, rate limiting, v.v.

- **Backend**: Bao gồm 7 dịch vụ riêng biệt, mỗi dịch vụ đảm nhiệm một chức năng chuyên biệt: Authentication, User, Gig, Order, Notification, Chat, và Review.
- **Data Storage**: Mỗi service (trừ Notification) đều có cơ sở dữ liệu riêng biệt, giúp đảm bảo tính độc lập và dễ mở rộng.



Hình 1.3: Sơ đồ hệ thống ShopMicro

Các dịch vụ trong hệ thống giao tiếp với nhau thông qua hai cơ chế chính:

- **RESTful API** sử dụng giao thức HTTP/HTTPS cho giao tiếp client-server.
- **Message Broker** sử dụng **RabbitMQ** để xử lý giao tiếp bất đồng bộ giữa các service, giảm độ phụ thuộc và tăng khả năng mở rộng.
- **Socket.IO** được tích hợp riêng cho các dịch vụ như Order và Chat để đảm bảo giao tiếp thời gian thực giữa người dùng và hệ thống.

4, Quy trình phát triển phần mềm theo mô hình Scrum.

4.1, Scrum là gì?

Scrum là một phương pháp agile tổng quát nhưng tập trung vào quản lý phát triển tăng dần (iterative development).

4.2, Quy trình phát triển phần mềm và vai trò của các quy trình theo mô hình Scrum

Trong quy trình phát triển phần mềm theo mô hình Scrum gồm có 3 quy trình:

- **Outline Planning và Architectural Design**
    - Thu thập và làm rõ các yêu cầu chức năng, phi chức năng của hệ thống.
    - Tạo danh sách (Product Backlog) các yêu cầu hoặc tính năng (User Story) mà sản phẩm cần có.
    - Xác định mô hình tổng thể, các module, giao diện, nền tảng, công nghệ và các quy ước kỹ thuật.
- **Sprint Cycle**
    - Điểm khởi đầu cho lập kế hoạch là lập danh sách các công việc cần thực hiện cho dự án.
    - **Select:** Team phát triển phải làm việc với khách hàng và chọn các tính năng và chức năng cần phát triển trong vòng sprint đó.
    - **Develop:** Team sẽ thực hiện thiết kế chi tiết, lập trình, kiểm thử, tích hợp, ...
    - **Review:** Kết thúc Sprint, nhóm tổ chức review, trình bày sản phẩm cho Product Owner và bên đặt hàng.
    - **Assess:** Nhóm tự tổ chức đánh giá quá trình làm việc của Sprint vừa qua. Sau đó vòng sprint tiếp theo sẽ bắt đầu.
- **Project Closure**
    - Khi các vòng Sprint đã tạo ra sản phẩm hoàn chỉnh theo yêu cầu, dự án sẽ kết thúc.

II, Cơ sở lý thuyết

1, Lý luận tổng quan kiến trúc phần mềm

Kiến trúc phần mềm là cấu trúc tổng thể của một hệ thống phần mềm, bao gồm các thành phần, mối quan hệ giữa chúng, cũng như nguyên lý thiết kế và định hướng kỹ thuật được áp dụng trong toàn bộ hệ thống. Việc lựa chọn kiến trúc phù hợp giúp:

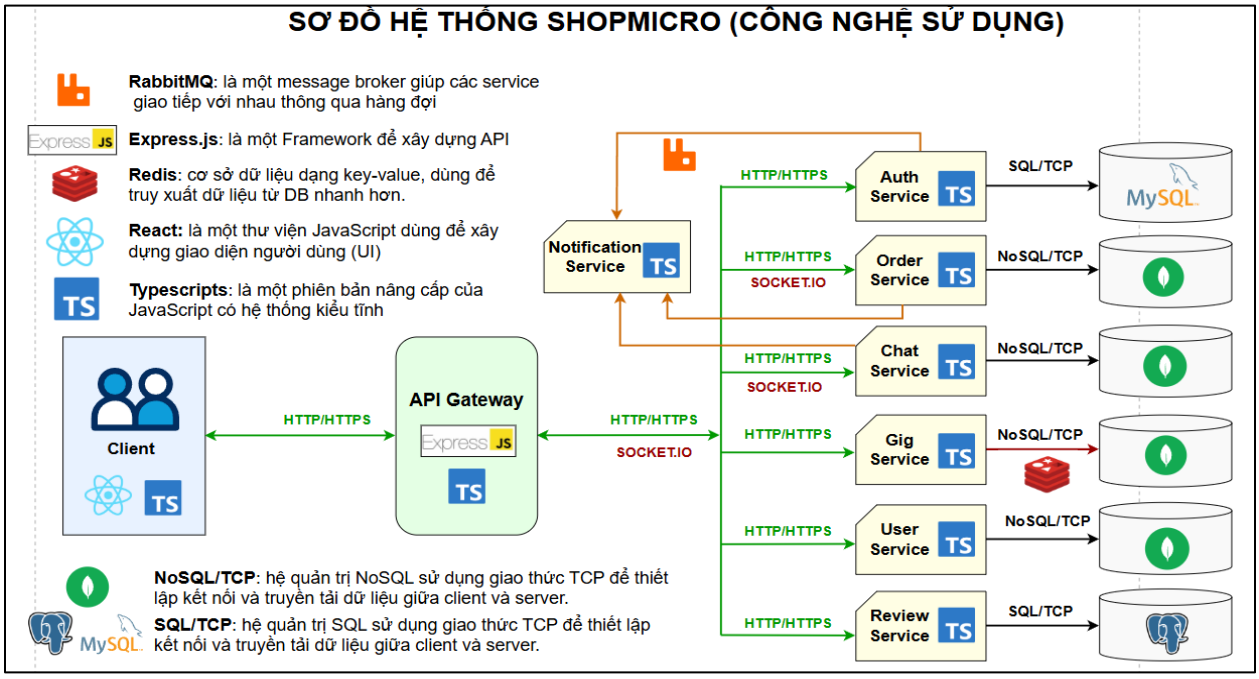
- Dễ bảo trì và mở rộng.
- Phân tách rõ ràng trách nhiệm các thành phần.
- Hỗ trợ kiểm thử và phát triển song song giữa các nhóm.

Hai nguyên lý quan trọng trong thiết kế phần mềm được hệ thống ShopMicro áp dụng là:

- **Single Responsibility Principle (SRP)**: Mỗi thành phần chỉ đảm nhận một trách nhiệm duy nhất.
- **Separation of Concerns (SoC)**: Phân chia mối quan tâm giữa các tầng (tầng truy cập dữ liệu, tầng dịch vụ, tầng điều khiển,...).

2, Công nghệ sử dụng

Về mặt vật lý triển khai, hệ thống được xây dựng và vận hành dựa trên các công nghệ và công cụ cụ thể như sau



Hình 2.2: Sơ đồ hệ thống ShopMicro (Công nghệ sử dụng)

- **Frontend**:
  - **React.js**: xây dựng giao diện người dùng hiện đại và linh hoạt.
  - **TailwindCSS**: thiết kế giao diện theo hướng tiện dụng, hỗ trợ responsive.
  - **TypeScript**: tăng tính ổn định và dễ bảo trì mã nguồn.
- **Backend**:
  - **Node.js + Express**: xử lý request nhanh chóng, phù hợp với hệ thống có nhiều kết nối đồng thời.
  - **TypeScript**: đảm bảo tính chặt chẽ trong phát triển backend.
  - Áp dụng mô hình N-layer như đã mô tả.
- **Giao tiếp và Hạ tầng**:

- **RabbitMQ**: đóng vai trò là Message Broker để giao tiếp giữa các microservices theo mô hình sự kiện (event-driven).
- **Socket.IO**: hỗ trợ giao tiếp thời gian thực cho các chức năng cần thiết như Chat và theo dõi đơn hàng.

- **Lưu trữ và Caching**:
  - **MySQL, PostgreSQL, MongoDB**: mỗi dịch vụ sử dụng một loại CSDL riêng biệt phù hợp với nhu cầu:
    - MySQL/PostgreSQL cho dữ liệu quan hệ (User, Order,...).
    - MongoDB cho dữ liệu phi quan hệ như thông tin linh hoạt trong Review hoặc Chat.
  - **Redis**: dùng để cache dữ liệu truy xuất nhiều lần, đồng thời hỗ trợ như một Message Queue trong một số tình huống nội bộ.
- **Logging & Monitoring**:
  - **Elasticsearch + Kibana**: thu thập, phân tích log và hỗ trợ tìm kiếm nâng cao.
- **Triển khai & Môi trường**:
  - Toàn bộ hệ thống được container hóa bằng **Docker**, mỗi service chạy trong một container riêng.
  - **Docker Compose** hoặc **Docker Swarm** được sử dụng để quản lý các container trong môi trường phát triển.
  - .env files và cấu hình riêng biệt cho từng service giúp dễ dàng tái sử dụng và bảo trì.

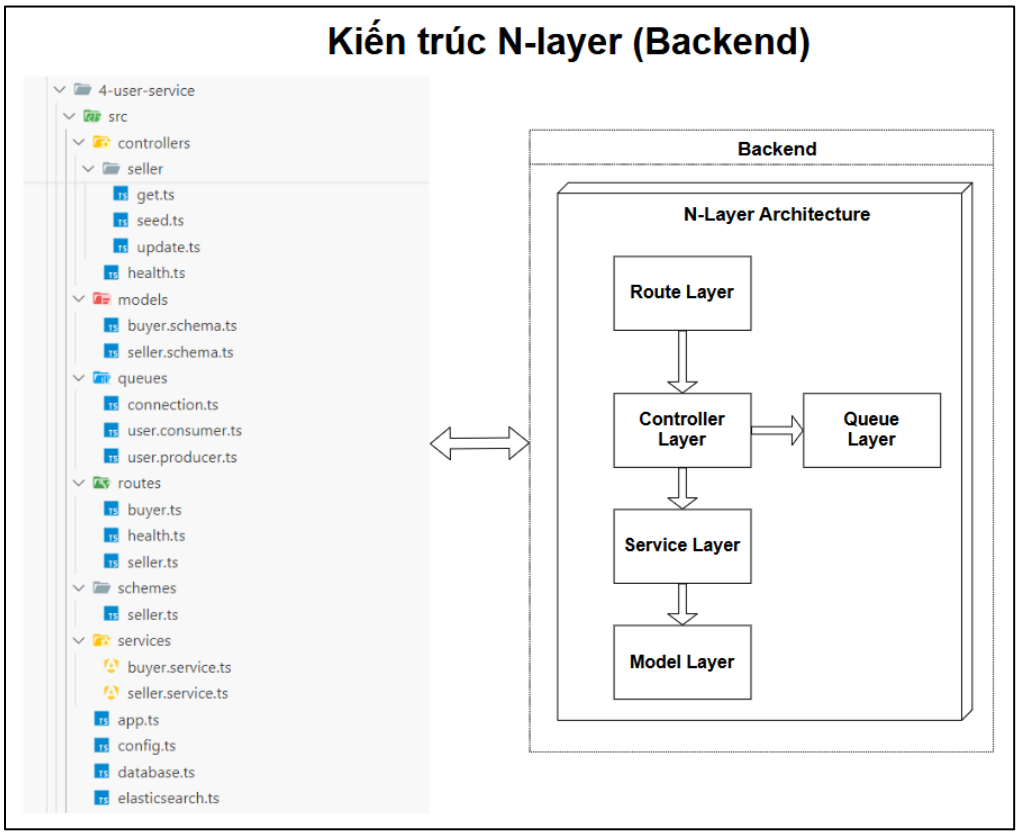
3, Kiến trúc Node.js (Back-End và Front-End)

Toàn bộ hệ thống ShopMicro được xây dựng trên nền tảng **Node.js**, trong đó:

- **Back-End** sử dụng kiến trúc **N-Layer (tầng)**, kết hợp với mô hình **Module hóa**.
- **Front-End** đơn giản, không theo kiến trúc đặc biệt, chỉ tập trung hiển thị và tương tác với người dùng thông qua API Gateway.

3.1, Phân tầng kiến trúc (N-Layer, Clean Architecture)

Toàn bộ dự án backend được tổ chức theo module, mỗi module tương ứng với một chức năng nghiệp vụ như: user-service, auth-service, gig-service, v.v. Mỗi module lại chia theo kiến trúc phân tầng như hình:



Hình 2.3.1: Kiến trúc N-layer (Backend)

Tầng	Mô tả
Route Layer	Xử lý các HTTP route như: POST /signin, PUT /verify-email, ...
Controller Layer	Xử lý logic ban đầu, gọi services, trả response, xử lý OTP, token, schema validation.
Queue Layer	Xử lý các message bất đồng bộ* giữa các Services qua RabbitMQ. *message bất đồng bộ ≈ giao tiếp
Service Layer	Giao tiếp với DB, xử lý logic nghiệp vụ. Ví dụ như: tạo user, xác thực, lấy OTP, v.v.

Model Layer	Định nghĩa schema cho dữ liệu trong MongoDB, MySQL và PostgreSQL.

3.2, Ví dụ minh họa

3.2.1, Controller (Tầng điều khiển)

```
const email = async (req: Request, res: Response): Promise<void>
=> {
  const buyer = await getBuyerByEmail(req.currentUser!.email);
  res.status(200).json({ message: 'Buyer profile', buyer });
};
```

3.2.2, Service (Tầng nghiệp vụ)

```
const getRandomBuyers = async (count: number):
Promise<IBuyerDocument[]> => {

  const buyers: IBuyerDocument[] = await
BuyerModel.aggregate([{ $sample: { size: count } }]);

  return buyers;
```

3.2.3, Queue (Tầng giao tiếp message queue):

```
channel.consume(queueName, async (msg) => {

  const { type, email } = JSON.parse(msg.content.toString());

  if (type === 'auth') {

    await createBuyer({ email, ... });
```

3.2.4, Route (Tầng xử lý API):

```
router.get('/email', email);
router.get('/username', currentUsername);});
```

3.2.5, Model (Tầng dữ liệu):

```
const buyerSchema = new Schema({
  username: { type: String, required: true },
  email: { type: String, required: true },
  ...
});
```

4, Frontend

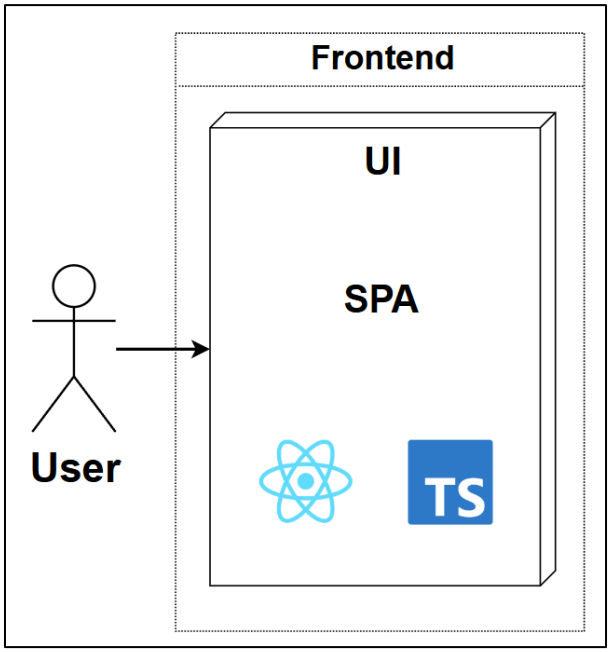
4.1, Công nghệ Frontend

Hệ thống MicroShop sử dụng React.js, TailwindCSS và TypeScript để phát triển giao diện người dùng.

- **React.js**: Là thư viện JavaScript phổ biến nhất hiện nay cho việc xây dựng giao diện người dùng động. React cho phép phát triển ứng dụng theo hướng component, giúp tái sử dụng mã, cải thiện hiệu năng thông qua Virtual DOM, và tối ưu trải nghiệm người dùng trong các ứng dụng thương mại điện tử phức tạp như MicroShop.

- **TailwindCSS**: Là một framework CSS tiện dụng với triết lý "utility-first", giúp phát triển giao diện nhanh chóng, sạch sẽ mà không cần viết quá nhiều CSS tùy chỉnh. Tailwind hỗ trợ responsive tốt, giúp tối ưu trải nghiệm người dùng trên cả desktop và thiết bị di động.

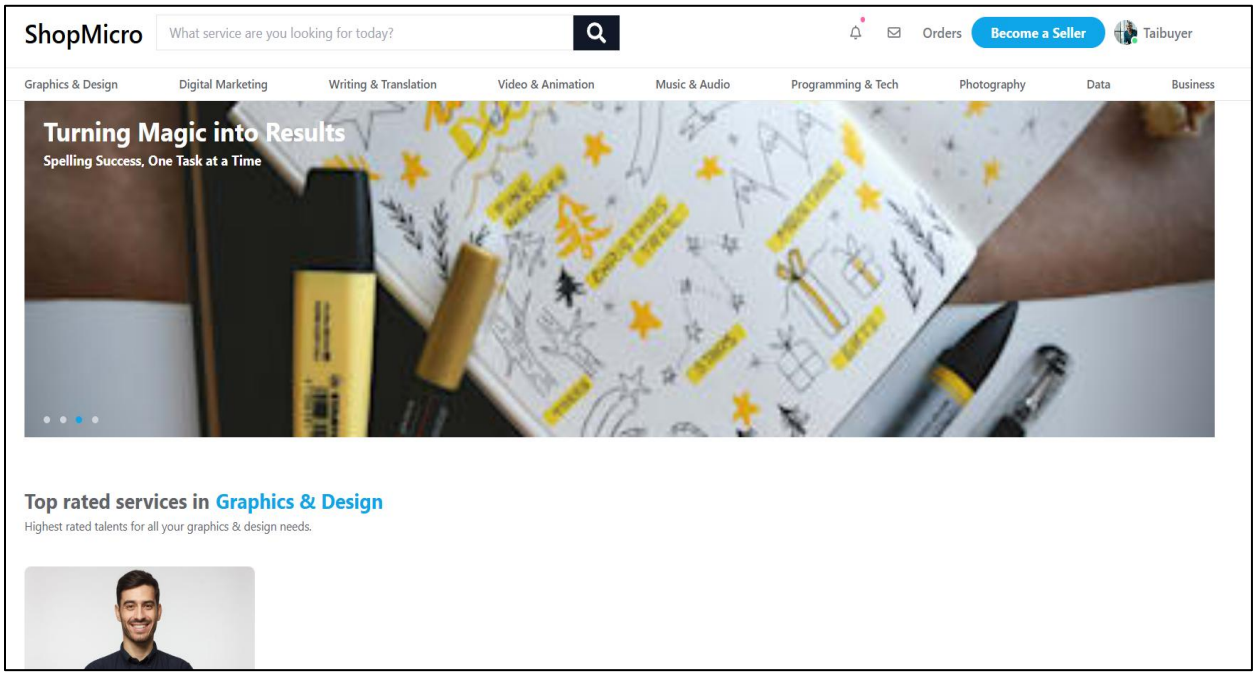
- **TypeScript**: Việc kết hợp TypeScript vào frontend giúp kiểm soát kiểu dữ liệu rõ ràng, giảm thiểu lỗi runtime, và hỗ trợ tốt hơn cho việc phát triển các component phức tạp. Điều này đặc biệt hữu ích khi làm việc trong team hoặc với quy mô codebase lớn.



Hình 2.4.1: Công nghệ - Frontend

4.2, Cơ chế SPA và định tuyến

Ứng dụng hoạt động theo mô hình SPA, sử dụng BrowserRouter để quản lý định tuyến. Mọi yêu cầu đến trình duyệt đều được xử lý bởi index.html, nơi React sẽ render lại giao diện tùy theo route được định nghĩa trong AppRouter.



Hình 2.4.2: Trang chủ ShopMicro

4.3, Cơ chế xử lý tương tác người dùng

- Các tương tác như click, submit, form,... được xử lý bằng React và các state hook.
- Các thông báo như thành công/thất bại hiển thị thông qua react-toastify.
- Kết nối socket với server cho các tính năng như nhắn tin thời gian thực được khởi tạo trong App.tsx:

```
useEffect(() => {
  socketService.setupSocketConnection();
}, []);
```

4.4, Triển khai frontend

- Frontend được đóng gói và phục vụ bằng Nginx (xem file nginx.conf), với cấu hình như sau:

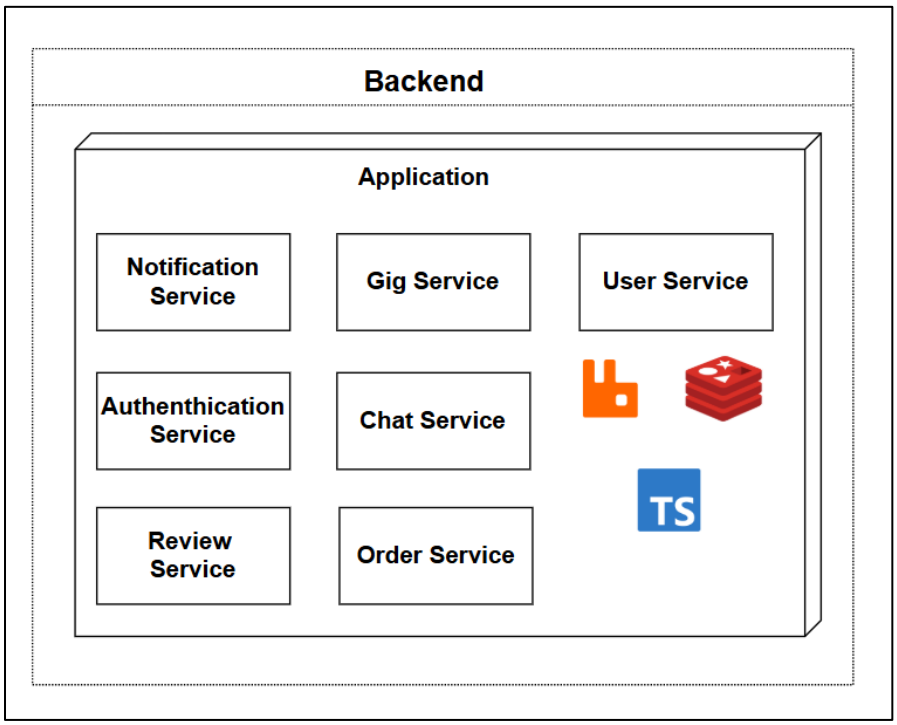
- Giao diện được phục vụ tại location /, mọi route không tồn tại sẽ fallback về index.html để React xử lý.
- Các tài nguyên tĩnh như .js, .css, .svg được cache.
- Bảo mật bằng cách chặn các source map (.js.map) và giới hạn các loại file.

5, Backend

5.1, Kiến trúc Backend

Phần backend của hệ thống được phát triển với **Node.js**, **TypeScript, RabbitMQ, Redis, Elasticsearch** và kiến trúc **N-layer**, được lựa chọn dựa trên các tiêu chí hiệu năng, khả năng mở rộng và tổ chức mã nguồn rõ ràng:

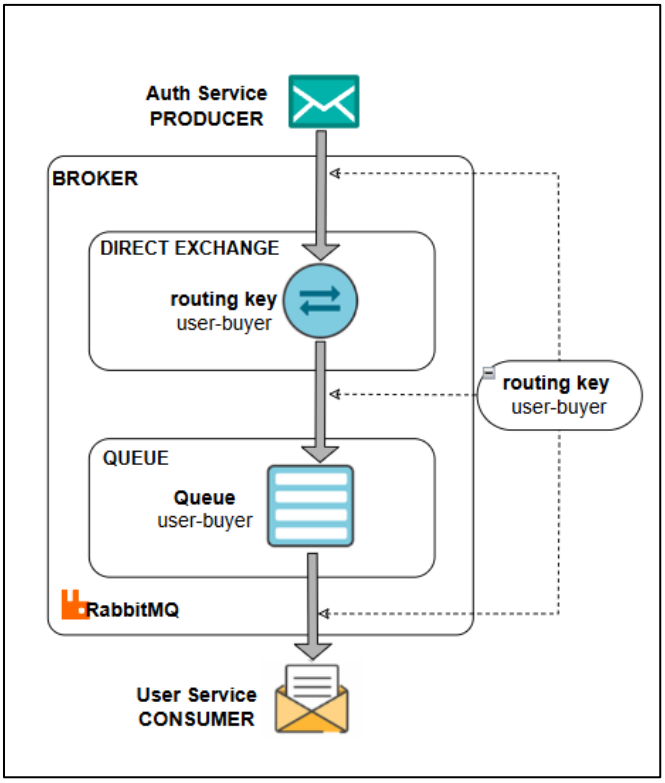
- **Node.js**: Cung cấp môi trường chạy JavaScript phía server với khả năng xử lý non-blocking và asynchronous, rất phù hợp với các hệ thống có nhiều kết nối đồng thời như một sàn thương mại điện tử. Ngoài ra, việc dùng cùng ngôn ngữ JavaScript ở cả frontend và backend giúp đồng bộ trong phát triển.
- R**abbitMQ**: Được sử dụng để **giao tiếp không đồng bộ** giữa các microservices thông qua mô hình message broker. Thay vì gọi trực tiếp lẫn nhau, các service gửi message đến hàng đợi, giúp hệ thống **giảm độ phụ thuộc**, dễ mở rộng và tăng khả năng chịu tải.
- **Redis**: Là hệ thống lưu trữ dạng key-value trong bộ nhớ, được sử dụng như **cache** để tăng tốc độ truy vấn dữ liệu thường xuyên sử dụng như: token, profile người dùng, session đăng nhập.
- **Mô hình N-layer (Controller – Service – Repository – Model)**: Được áp dụng nhằm tách biệt các tầng xử lý logic nghiệp vụ, thao tác cơ sở dữ liệu và điều hướng request, giúp mã nguồn rõ ràng, dễ kiểm thử, dễ mở rộng và bảo trì.



Hình 2.5.1: Công nghệ - Backend

5.2, Giao tiếp Microservices bằng RabbitMQ

Các microservices trong hệ thống sử dụng RabbitMQ làm message broker để trao đổi dữ liệu và thực hiện các tác vụ bất đồng bộ. Điều này giúp giảm phụ thuộc giữa các service và tăng khả năng mở rộng.



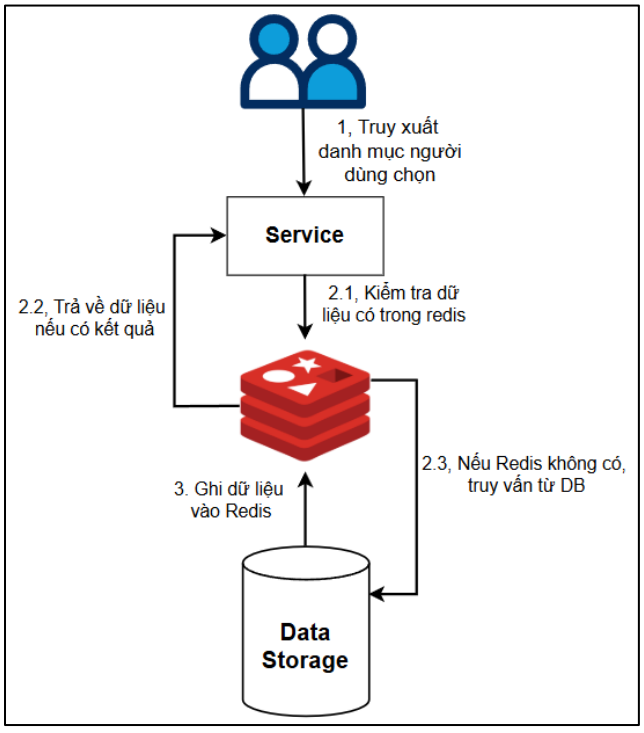
Hình 2.5.2: Message Broker - RabbitMQ

- **Producer:** Auth Service gửi message đến Exchange kèm routing-key "user-buyer".
- **Exchange (Direct):** Bộ định tuyến của RabbitMQ sẽ kiểm tra routing-key sẽ thuộc queue nào thì định tuyến đến queue đó.
- **Queue:** Là nơi chứa message queue và xử lý queue để gửi đến Consumer có routing-key tương ứng.
- **Consumer:** User Service nhận và xử lý message.

5.3, Caching dữ liệu với Redis

Cách tối ưu truy vấn bằng Redis:

- Client gửi request đến Service.
- Service kiểm tra dữ liệu có trong Redis hay không.
- Nếu Redis có dữ liệu => trả về kết quả ngay.
- Nếu Redis không có:
    - Truy vấn Database của service đó.
    - Ghi dữ liệu vào Redis.
    - Trả về kết quả cho client.

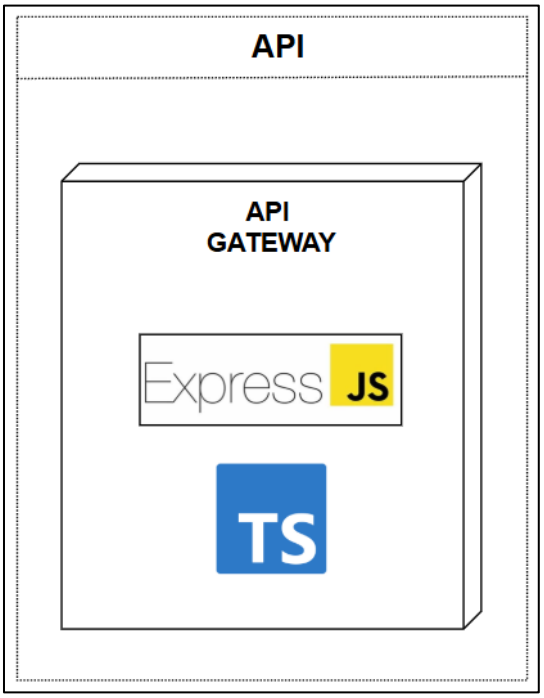


Hình 2.5.3: Caching Data - Redis

6, API Gateway

API Gateway là điểm vào trung tâm của toàn hệ thống, chịu trách nhiệm tiếp nhận request từ client, xác thực người dùng, định tuyến request đến các dịch vụ tương ứng, quản lý cache và xử lý các lỗi hệ thống.

Express.js: Là framework web tối giản, nhưng mạnh mẽ, giúp định nghĩa rõ ràng các route và xử lý request một cách hiệu quả. Express rất phù hợp cho kiến trúc microservices do tính nhẹ và dễ tích hợp với các thư viện khác như RabbitMQ, Redis.



Hình 2.6: Công nghệ - API Gateway

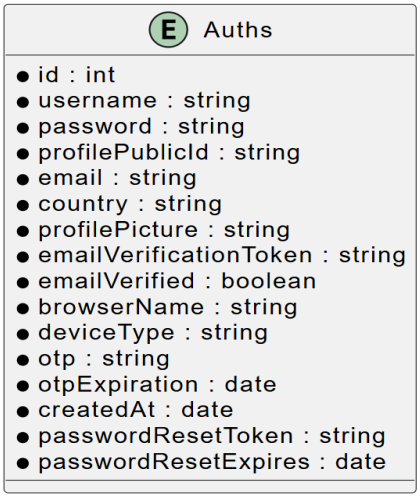
7, Data Storage

7.1, Cơ sở dữ liệu quan hệ – MySQL

- **Lý do sử dụng:**

MySQL được sử dụng trong dịch vụ xác thực (Authentication) bởi vì dữ liệu người dùng đăng ký đăng nhập có cấu trúc rõ ràng, quan hệ giữa các trường được định nghĩa chặt chẽ. MySQL cung cấp cơ chế kiểm soát ràng buộc, chỉ số, và đồng nhất dữ liệu – đặc biệt phù hợp với các hệ thống yêu cầu độ chính xác cao như xác thực người dùng.

- **Thiết kế dữ liệu (Lược đồ logic):**



Hình 2.7.1: Lược đồ Logic - MySQL

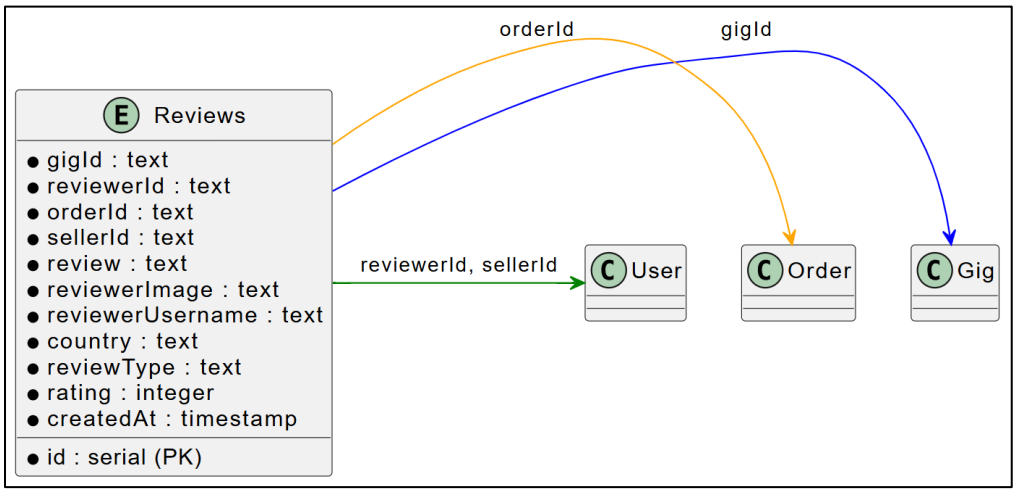
- **Quản lý truy cập và ORM**:
  - ORM: **Sequelize** (hỗ trợ mô hình Database First và Code First)
  - Connection: qua đối tượng sequelize
  - Hash mật khẩu sử dụng bcryptjs, tích hợp Hook beforeCreate
  - Các thao tác CRUD: AuthModel.create, AuthModel.findOne, AuthModel.update, AuthModel.sync

7.2, Cơ sở dữ liệu quan hệ – PostgreSQL

- **Lý do sử dụng:**

PostgreSQL được sử dụng để lưu trữ các đánh giá (reviews) vì nó hỗ trợ các tính năng mở rộng, hiệu năng cao trong việc lưu trữ dữ liệu dạng text, sắp xếp theo chỉ mục, và xử lý phức tạp như tìm kiếm theo nhiều tiêu chí (gigId, sellerId).

- **Thiết kế dữ liệu (Lược đồ logic):**



Hình 2.7.2: Lược đồ logic - PostgreSQL

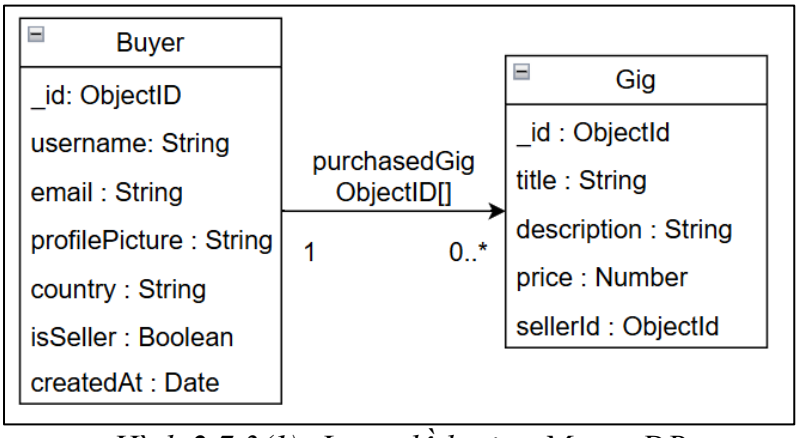
- **Quản lý truy cập**:
    - Sử dụng thư viện pg (PostgreSQL client for Node.js)
    - Connection: Pool từ pg
    - Thiết lập bảng qua truy vấn SQL thủ công trong databaseConnection()
    - Có hỗ trợ SSL khi chạy trên AWS Cluster

7.3, Cơ sở dữ liệu phi quan hệ – MongoDB

- **Lý do sử dụng:**

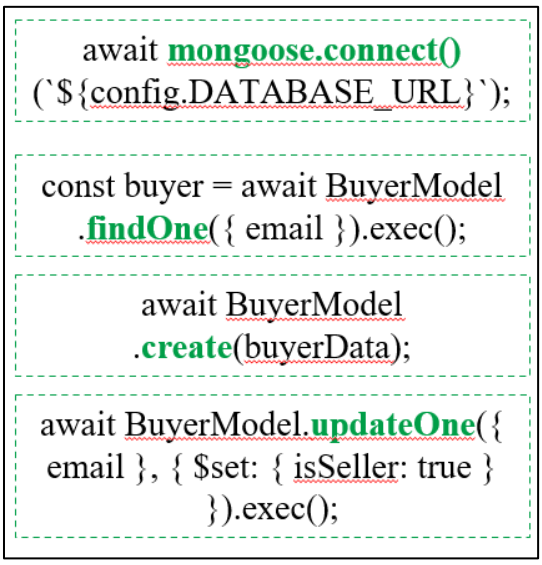
MongoDB được chọn cho các dịch vụ User, Gig, Chat, Order vì dữ liệu trong các dịch vụ này linh hoạt, có thể thay đổi theo thời gian, và không yêu cầu ràng buộc khóa ngoại chặt chẽ. MongoDB hỗ trợ tốt mô hình tài liệu, lưu trữ các cấu trúc phức tạp, và truy vấn hiệu quả bằng aggregation pipeline.

- **Thiết kế dữ liệu (Lược đồ logic)**



Hình 2.7.3(1): Lược đồ logic - MongoDB

- **Data Access – Connection và CRUD**
    - **Connection:** Kết nối MongoDB bằng mongoose.connect() với URL lấy từ biến môi trường (config.DATABASE_URL) thông qua service.
    - **Lấy dữ liệu:** Lấy một người dùng đã có sẵn trong DB với findOne()
    - **Tạo mới:**Tạo mới người dùng với create(buyerData) nếu email chưa tồn tại trong hệ thống.
    - **Cập nhật:** Cập nhật thuộc tính isSeller cho người mua bằng $set với điều kiện là email.



Hình 2.7.3(2): Data Access - MongoDB

III, Phân tích hệ thống

1, Business Context

Website MicroShop là một website freelance thương mại điện tử có các chức năng cơ bản như: dịch vụ **Thông báo**, dịch vụ **Xác thực**, dịch vụ **Người dùng**, dịch vụ **Sản phẩm**, dịch vụ **Trò chuyện**, dịch vụ **Đơn hàng** và dịch vụ **Đánh giá**.

Với dịch vụ **Xác thực**, người sử dụng hệ thống (khách hàng hoặc người bán) sẽ có những chức năng sau:

- **Đăng ký**
  - Người dùng đăng ký tài khoản.
- **Đăng nhập**
  - Người dùng nhập email và mật khẩu đã đăng ký để đăng nhập. Hệ thống kiểm tra thông tin, nếu hợp lệ sẽ cấp quyền truy cập vào hệ thống.
- **Xác thực tài khoản**
  - Người dùng phải xác thực tài khoản sau khi đăng nhập. Nếu không xác thực tài khoản thì người dùng sẽ bị bạn chế truy cập một số chức năng trong hệ thống.
- **Gửi lại email xác thực tài khoản**

    - Nếu người dùng đăng nhập mà email xác thực hết hạn hoặc email gửi bị lỗi, người dùng có thể nhấn và “Resend Verify Email” để nhận lại email xác thực lại tài khoản.
- **Quên mật khẩu**
    - Khi quên mật khẩu, người dùng bấm vào "Quên mật khẩu" trên giao diện đăng nhập.
    - Người dùng sẽ được chuyển đến trang nhập email đã đăng ký để lấy lại mật khẩu.
    - Hệ thống sẽ gửi đến email người dùng nhập và tiến hành đổi mật khẩu.
- **Tìm kiếm sản phẩm**
    - Người dùng tìm kiếm sản phẩm bằng tên sản phẩm, thời gian giao hàng, khoảng giá.

Với dịch vụ **Người dùng**, người sử dụng hệ thống sẽ có những chức năng sau:

- **Trở thành người bán hàng**
    - Sau khi đăng ký thì người dụng mặc định là người mua hàng. Nếu người mua hàng muốn trở thành người bán thì nhấn vào “Become a seller”, người mua hàng cập nhật hồ sơ của mình thành người bán hàng.
- **Quản lý profile người bán**
    - Người bán có thể cập nhật hồ sơ của mình.
    - Cập nhật số lượng công việc
    - Cập nhật số lượng công việc đang làm
    - Cập nhật số lượng công việc đã hoàn thành
    - Cập nhật số lượng công việc bị hủy

Với dịch vụ **Thông báo,** sẽ quản lý các email thông báo gửi đến dịch vụ khác như dịch vụ **Xác thực, Trò chuyện** và **Đơn hàng.**

- Đối với dịch vụ **Xác thực** thì sẽ có email thông báo “Xác thực tài khoản”, “Quên mật khẩu” và “Đổi mật khẩu”.
- Đối với dịch vụ **Trò chuyện**, có thông báo trò chuyện real-time.
- Dịch vụ **Đơn hàng** sẽ có 5 email gửi cho 5 trường hợp sau:

- Email 1 là email offer đơn hàng. Người bán sẽ offer một đơn hàng cho khách hàng.
- Email 2 là email đặt hàng. Nếu người mua đặt hàng thì sẽ có email thông báo gửi đến người bán.
- Email 3 là gửi email thông báo hóa đơn mua hàng cho người mua khi người mua đã thanh toán đơn hàng.
- Email 4 là gửi email thông báo yêu cầu chờ hàng khi người bán hàng cần thêm thời gian chuẩn bị đơn hàng.
- Email 5 là gửi email thông báo xác nhận yêu cầu chờ hàng tới khách hàng, khách hàng sẽ chấp nhận hoặc từ chối yêu cầu chờ hàng của người bán.
- Email 6 là gửi email thông báo đơn hàng đã được gửi đi từ người bán hàng.

Với dịch vụ **Sản phẩm**, người sử dụng hệ thống sẽ có những chức năng sau:

- **Quản lý sản phẩm**
  - Đối với người bán hàng sẽ có chức năng quản lý sản phẩm như thêm/xóa/sửa sản phẩm.

Với dịch vụ **Trò chuyện**, người bán hàng và khách hàng sẽ trò chuyện với nhau để trao đổi mua bán.

- **Trò chuyện real-time**
  - Khách hàng tìm thấy sản phẩm mà họ quan tâm và nhấp vào nút “Liên hệ”, hệ thống hiển thị một chat box nhỏ để trò chuyện với người bán, khi khách hàng gửi tin nhắn trong chat box thì người bán hàng sẽ nhận được tin nhắn ngay lập tức.
  - Mục đích của dịch vụ Trò chuyện là khách hàng liên hệ với người bán để tạo ra một sản phẩm mà khách hàng cần.
- **Đánh dấu đã đọc tin nhắn**
  - Người dùng đánh dấu tin nhắn đã đọc (1 hoặc nhiều tin nhắn)
- **Offer sản phẩm (Người bán)**
  - Người bán có thể “Offer” sản phẩm được hoành thành trong một khoảng thời gian đề xuất với khách hàng trong trang tin nhắn.

Với dịch vụ **Đơn hàng,** khách hàng có thể mua hàng bằng cách liên hệ với người bán thông qua dịch vụ **Trò chuyện**.

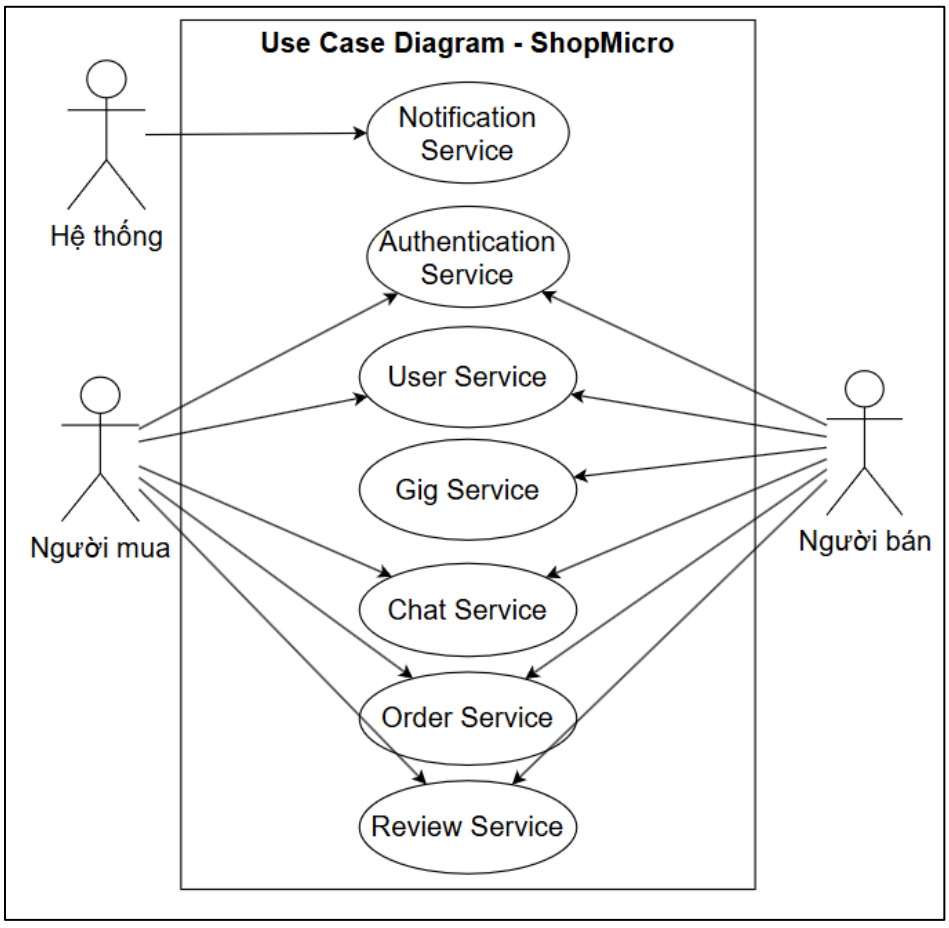
- **Đặt đơn hàng – Người mua**
    - Khi khách hàng nhận được một email "Offer đơn hàng" từ người bán.
    - Khách hàng chọn "Accept Offer" từ email thông báo của người bán thì khách hàng được chuyển sang trang thánh toán.
    - Sau khi khách hàng "Accept Offer", người bán nhận được một email "Đặt hàng" từ khách hàng.
- **Thanh toán đơn hàng – Người mua**
    - Khách hàng tiến hành thanh toán đơn hàng. Thanh toán thành công, khách hàng nhận được một email "Hóa đơn".
    - Thanh toán thành công hệ thống hiển thị "Start Order", khách hàng nhấp vào "Start Order" để đơn hàng bắt đầu được thực hiện từ người bán hàng.
- **Chấp nhận/từ chối thời gian chuẩn bị đơn hàng – Người bán**
    - Khách hàng sẽ nhận được email "Thêm thời gian chuẩn bị đơn hàng" từ người bán.
    - Khách hàng có thể chấp nhận/từ chối yêu cầu.
- **Yêu cầu thêm thời gian chuẩn bị đơn hàng – Người bán**
    - Khi người bán hàng chưa chuẩn bị xong đơn hàng trước dead-line được đề xuất thì người bán nhấp vào "Extend Delivery Date" để yêu cầu khách hàng thêm thời gian chuẩn bị đơn hàng.
    - Người bán sẽ nhận được email "Chấp nhận/từ chối thêm thời gian chuẩn bị đơn hàng" từ khách hàng.
- **Tạo đơn hàng mới – Người bán**
    - Người bán tạo đơn hàng mới với người mua
    - Email thông báo "Người bán đã đặt hàng sản phẩm của bạn" gửi đến người mua.
- **Giao hàng – Người bán**
    - Khi người bán chuẩn bị xong đơn hàng thì người bán nhấp vào "Delivery Now" để thực hiện gửi đơn hàng cho khách hàng.

- Khách hàng sẽ nhận được email “Giao hàng” từ người bán hàng.

Với dịch vụ **Đánh giá**

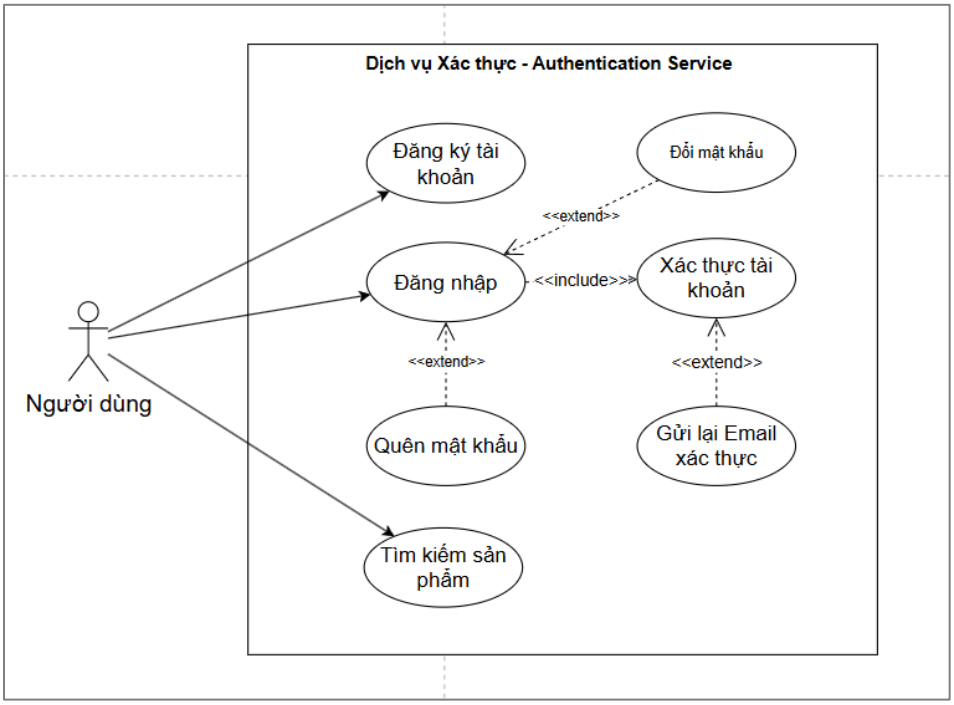
- Đánh giá & xếp hạng
  - Khách hàng có để lại bình luận đánh giá và mức độ hài lòng đối với đơn hàng.
  - Người bán có thể phản hồi lại bình luận đánh giá của khách hàng.

2, Use Case Diagram



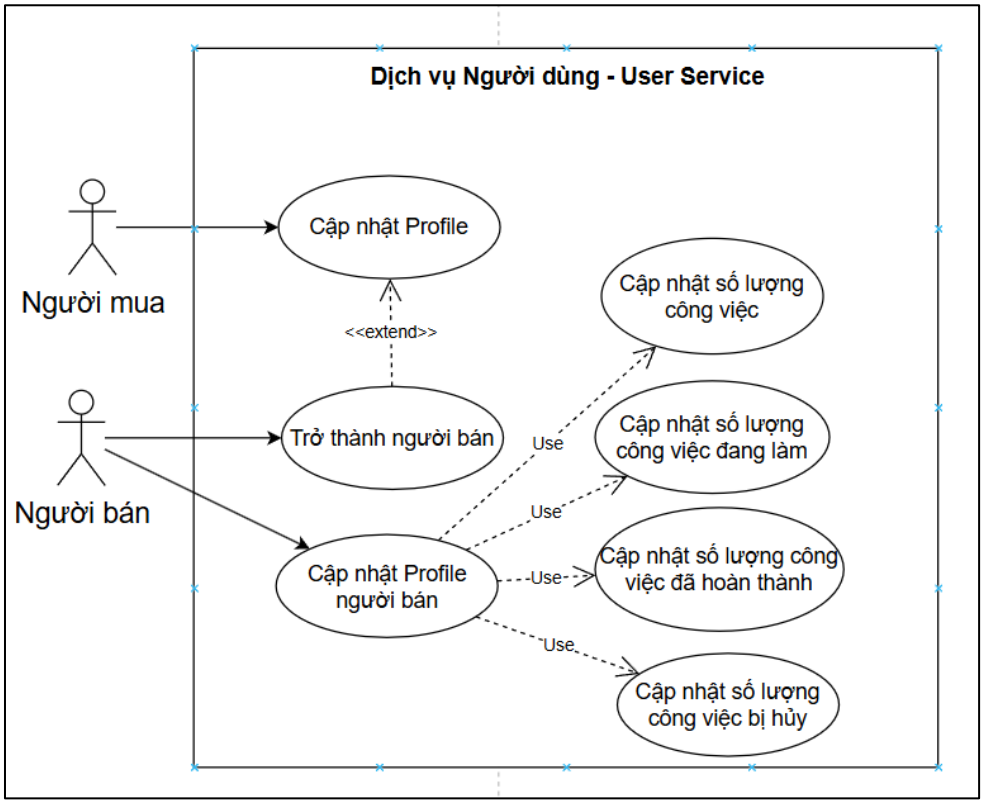
Hình 3.2: Use Case Diagram - ShopMicro

2.1, UC01: Dịch vụ Xác thực



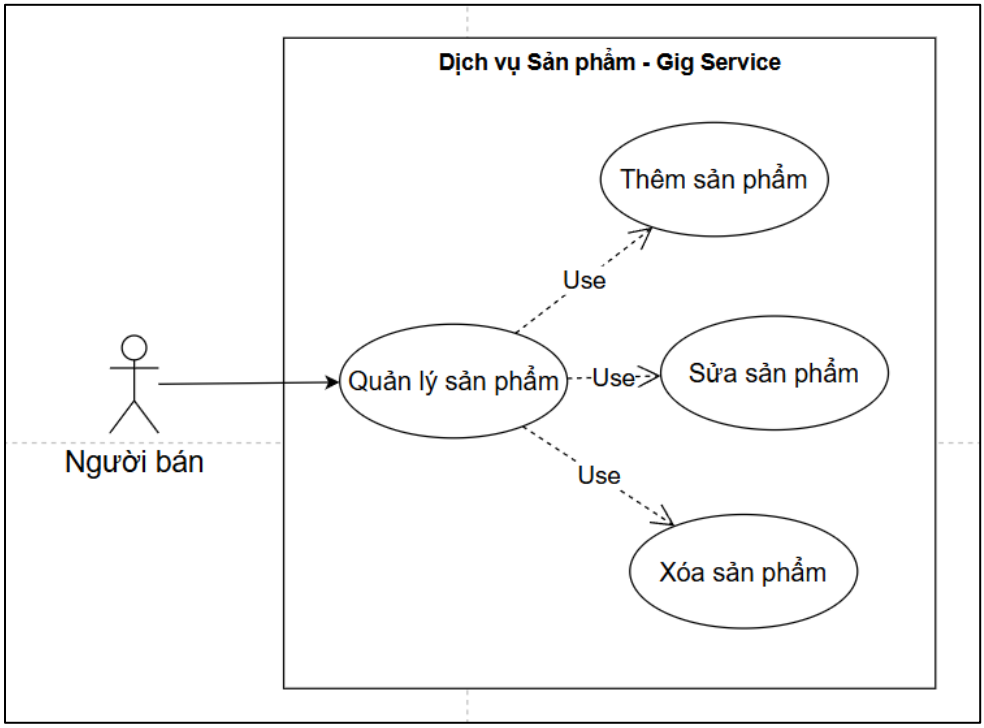
Hình 3.2.1 – UC01: Dịch vụ xác thực

2.2, UC02: Dịch vụ Người dùng



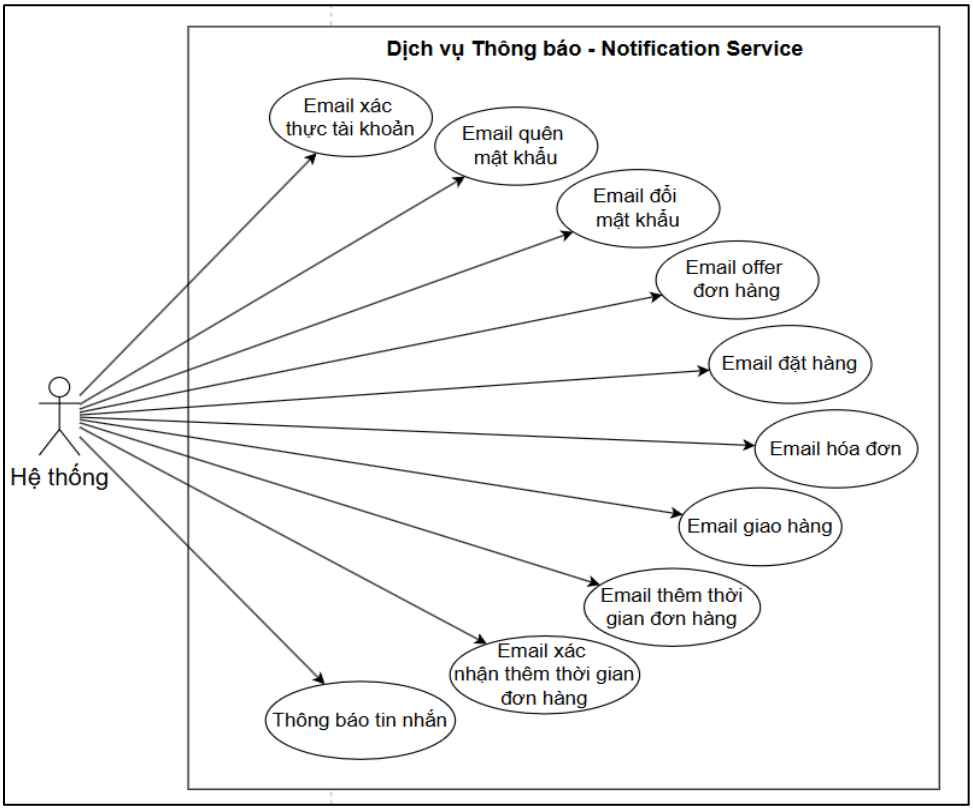
Hình 3.2.2 – UC02: Dịch vụ người dùng

2.3, UC03: Dịch vụ sản phẩm



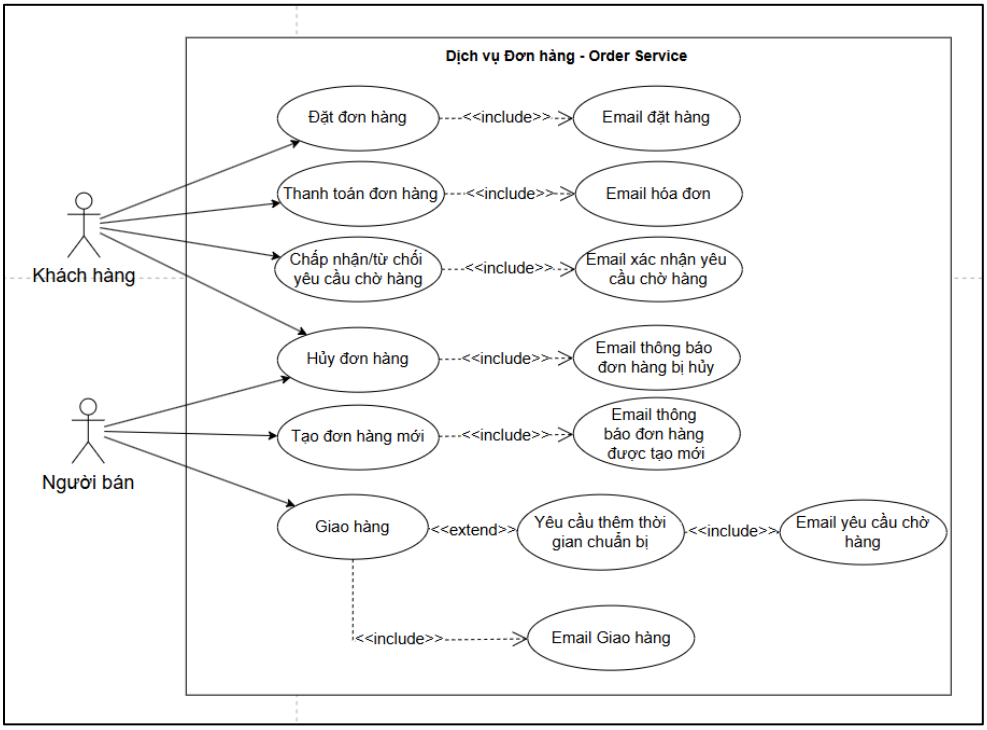
Hình 3.2.3 – UC03: Dịch vụ sản phẩm

2.4, UC04: Dịch vụ Thông báo



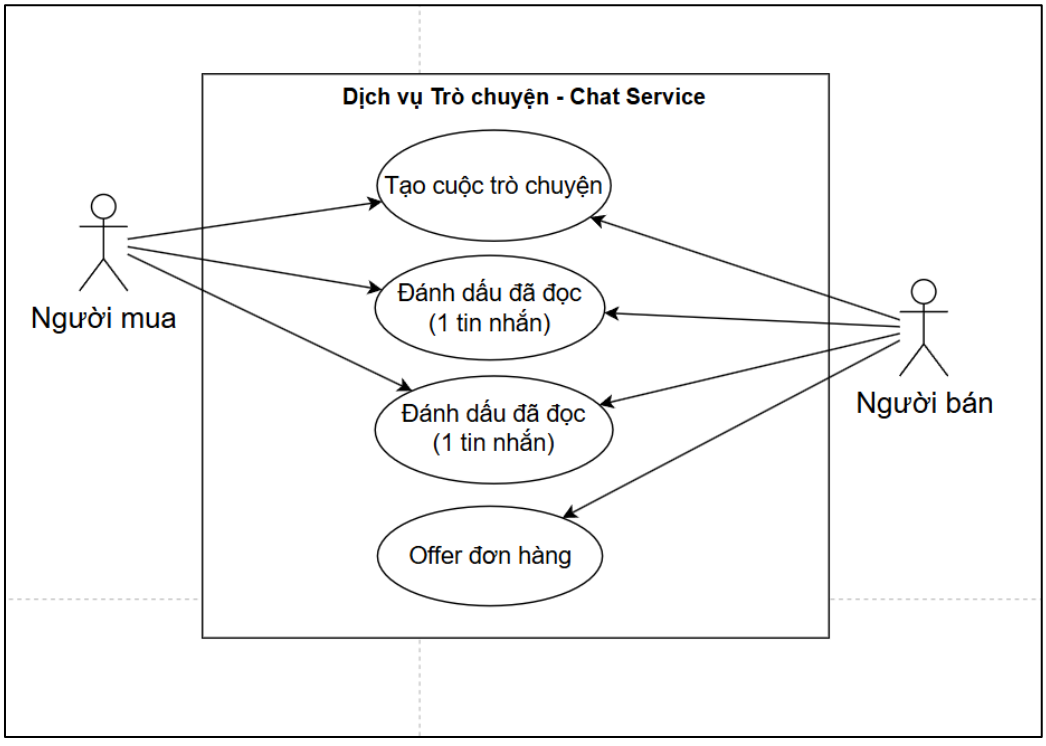
Hình 3.2.4 – UC04: Dịch vụ thông báo

2.5, UC05: Dịch vụ Đơn hàng



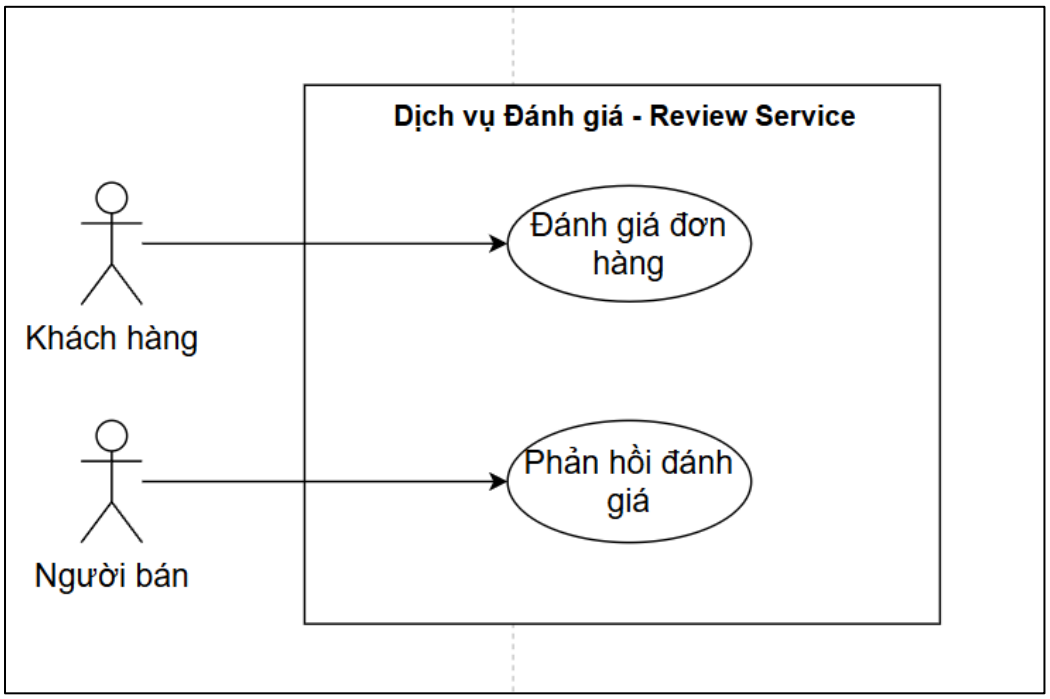
Hình 3.2.5 – UC05: Dịch vụ đơn hàng

2.6, UC06: Dịch vụ trò chuyện



Hình 3.2.6 – UC06: Dịch vụ trò chuyện

2.7, UC07: Dịch vụ đánh giá



Hình 3.2.7 – UC07: Dịch vụ đánh giá

3, User Story

3.1, Xác thực & Truy cập của người dùng

3.1.1, Đăng ký tài khoản người dùng

- User Story:
  - **Với** tư cách là người dùng mới.
  - **Tôi muốn** đăng ký tài khoản (theo cách "Sign Up" cho khách hàng hoặc "Become a Seller" cho người bán).
  - **Để** có thể sử dụng các chức năng của website.

3.1.2, Đăng nhập

- User Story:
  - **Với** tư cách là người dùng đã đăng ký (Khách hàng/Người bán).
  - **Tôi muốn** đăng nhập bằng email và mật khẩu.
  - **Để** truy cập vào tài khoản của mình.

3.1.3, Quên mật khẩu

- User Story:
  - **Với** tư cách là người dùng.

- **Tôi muốn** có chức năng "Quên mật khẩu".
- **Để** lấy lại mật khẩu khi không nhớ để đảm bảo tôi luôn có thể truy cập tài khoản.

3.1.4, Đổi mật khẩu

- User Story:
  - **Với** tư cách là người dùng (Khách hàng/Người bán).
  - **Tôi muốn** thay đổi mật khẩu từ liên kết email "Quên mật khẩu"
  - **Để** tôi có thể bảo mật tài khoản của mình sau khi đặt lại mật khẩu.

3.2, Quản lý người dùng & Dashboard

3.2.1, Quản lý người dùng

- User Story:
  - **Với** tư cách là người dùng (Khách hàng/Người bán).
  - **Tôi muốn** cập nhật thông tin Profile của mình.
  - **Để** đảm bảo dữ liệu cá nhân luôn được cập nhật và chính xác.

3.2.2, Lịch sử đơn hàng

- User Story:
  - **Với** tư cách là khách hàng.
  - **Tôi muốn** xem lịch sử đơn hàng của mình.
  - **Để** tôi có thể theo dõi các giao dịch đã mua và trạng thái đơn hàng của mình.

3.2.3, Dashboard người bán hàng

- User Story:
  - **Với** tư cách là người bán hàng.
  - **Tôi muốn** xem dashboard bán hàng của mình.
  - **Để** tôi có thể theo dõi sản phẩm, đơn hàng và đánh giá của khách hàng.

3.3, Quản lý sản phẩm & trò chuyện

3.3.1, Xem danh mục & Tìm kiếm sản phẩm

- User Story:

- **Với** tư cách là khách hàng.
- **Tôi muốn** xem danh mục sản phẩm và sử dụng chức năng tìm kiếm.
- **Để** tôi có thể nhanh chóng tìm ra sản phẩm phù hợp với nhu cầu của mình.

3.3.2, Trò chuyện real-time giữa khách hàng và người bán

- User Story:
  - **Với** tư cách là người bán hàng.
  - **Tôi muốn** xem dashboard bán hàng của mình.
  - **Để** tôi có thể theo dõi sản phẩm, đơn hàng và đánh giá của khách hàng.

3.3.3, Offer sản phẩm qua chat

- User Story:
  - **Với** tư cách là người bán hàng.
  - **Tôi muốn** gửi "Offer" sản phẩm cùng với thời gian hoàn thành dự kiến qua chat.
  - **Để** tôi có thể theo dõi sản phẩm, đơn hàng và đánh giá của khách hàng.

3.4, Xử lý đơn hàng

3.4.1, Offer sản phẩm qua chat

- User Story:
  - **Với** tư cách là người bán.
  - **Tôi muốn** gửi "Offer" sản phẩm cùng với thời gian hoàn thành dự kiến qua chat.
  - **Để** khách hàng có thể cân nhắc và chốt giao dịch.

3.4.2, Đặt đơn hàng

- **User Story:**
  - **Với** tư cách là khách hàng.
  - **Tôi muốn** "Accept Offer" từ email thông báo và chuyển sang trang thanh toán.
  - **Để** hoàn tất đơn hàng.

3.4.3, Thanh toán đơn hàng

- **User Story:**
  - **Với** tư cách là khách hàng.

- **Tôi muốn** thanh toán đơn hàng một cách an toàn.
- **Để** nhận được hóa đơn và kích hoạt quá trình thực hiện đơn hàng.

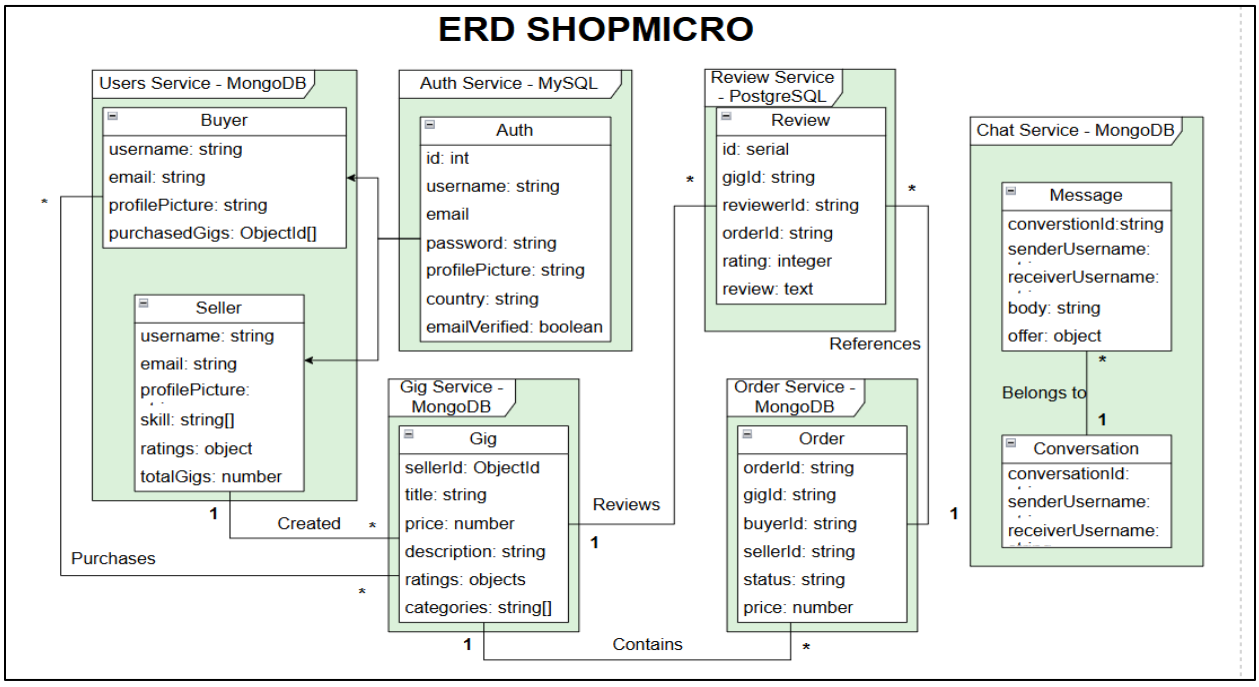
3.4.4, Yêu cầu thêm thời gian chuẩn bị đơn hàng

- **User Story:**
  - **Với** tư cách là người bán.
  - **Tôi muốn** yêu cầu thêm thời gian chuẩn bị đơn hàng khi cần.
  - **Để** đảm bảo đơn hàng được xử lý đúng hạn và đúng yêu cầu của khách hàng dù có sự chậm trễ.

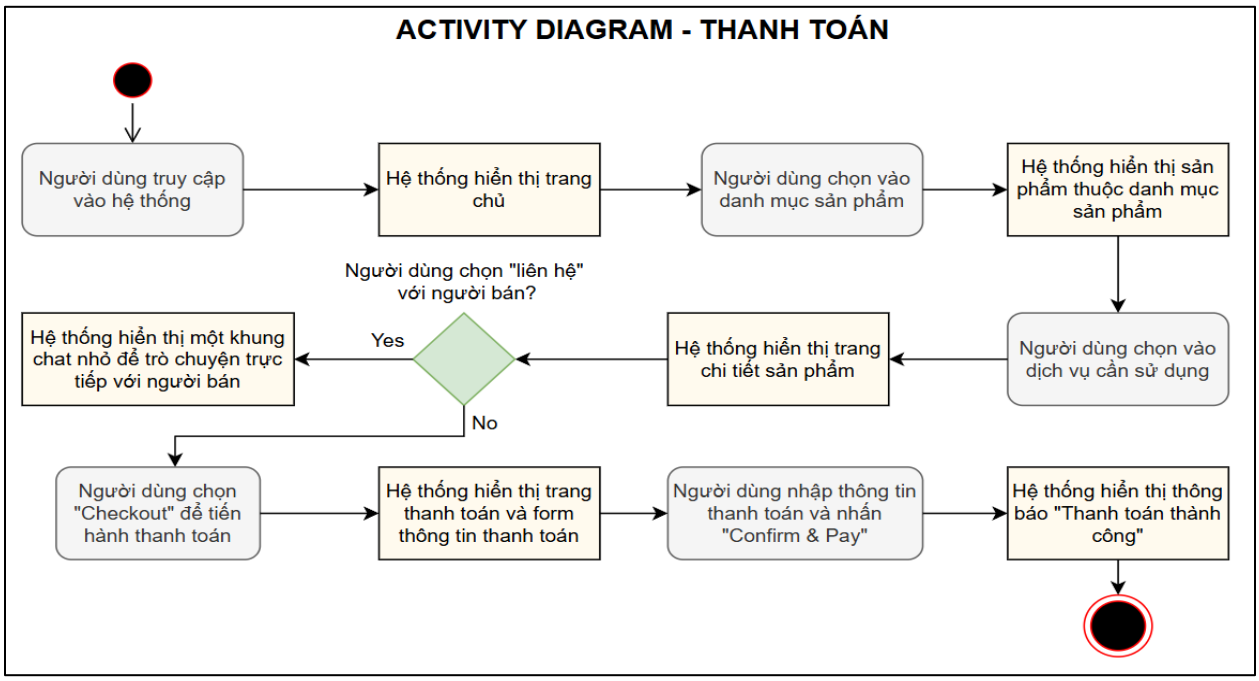
3.4.5, Giao hàng

- **User Story:**
  - **Với** tư cách là người bán.
  - **Tôi muốn** đánh dấu đơn hàng đã được giao bằng cách nhấp "Delivery Now".
  - **Để** khách hàng nhận được đơn hàng và thông báo về trạng thái giao hàng.

4, Lược đồ dữ liệu mức khái niệm



Hình 3.4(1): ERD ShopMicro



Hình 3.4(2): Activity Diagram – Thanh toán

IV, Thiết kế hệ thống

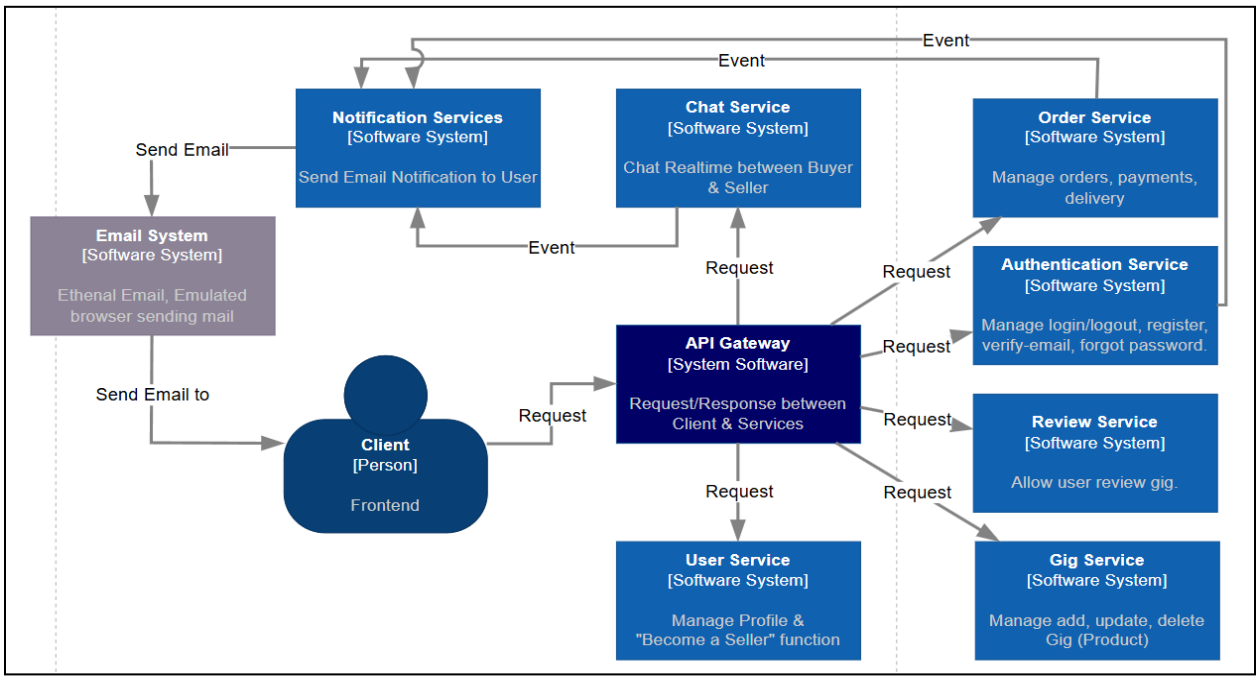
1, C4 Model

C4 Model là một cách tiếp cận trực quan để mô tả kiến trúc phần mềm ở 4 cấp độ chi tiết: Context → Container → Component → Code.

1.1, System Context Model

System Context diagram là sơ đồ giúp người xem hiểu hệ thống phần mềm đang làm gì, ai tương tác với nó, và liên kết với hệ thống nào khác. Cung cấp bức tranh tổng thể cho người không chuyên về kỹ thuật.

- Xác định rõ ranh giới của hệ thống.
- Giúp hiểu ai sử dụng hệ thống và những hệ thống nào tương tác với nó.



Hình 4.1.1 – System Context Diagram

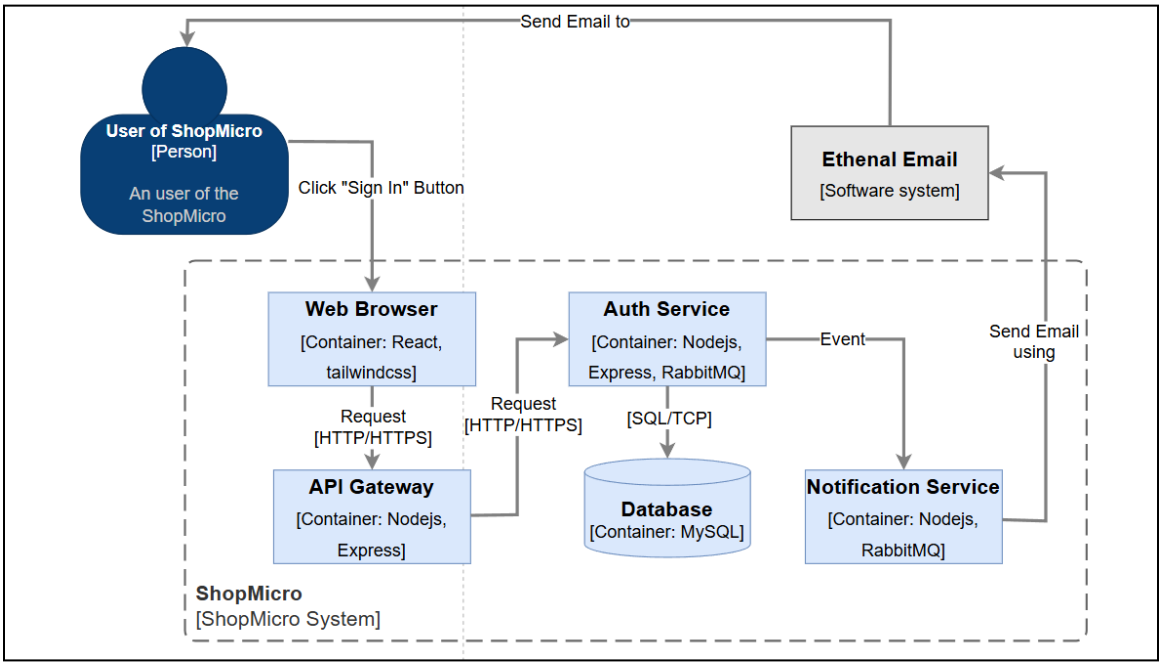
- **Giải thích sơ đồ:**
  - Client: là người dùng, sẽ tương tác với hệ thống.
  - API Gateway: là "trung tâm" tiếp nhận request giữa client và các Services.
  - Services: Là hệ thống được chia nhỏ ra thành các dịch vụ.
  - Database: Là thành phần lưu trữ dữ liệu tương ứng cho từng services.
  - Request/Response: Mối liên hệ giữa các thực thể.
- **Giải thích cách hoạt động:**
  - Request bắt đầu từ phía client và gửi đến API Gateway
  - API Gateway có nhiệm vụ xác định request và định tuyến request đến đúng Service tương ứng của request đó.
  - Service sẽ xử lý logic của request này và response về phía client.

1.2, Container Diagram

Container Diagram là một sơ đồ thể hiện kiến trúc phần mềm ở mức độ tổng quan và cách các chức năng được phân chia giữa các container. Giúp các developer và kiến trúc sư hiểu được kiến trúc tổng thể của hệ thống ở cấp triển khai logic.

- Mô tả cách các container giao tiếp với nhau thông qua các giao thức như HTTP/HTTPS,...

- Hiển thị công nghệ sử dụng trong mỗi container.



Hình 4.1.2 – Container Diagram (Ex: Chức năng đăng nhập)

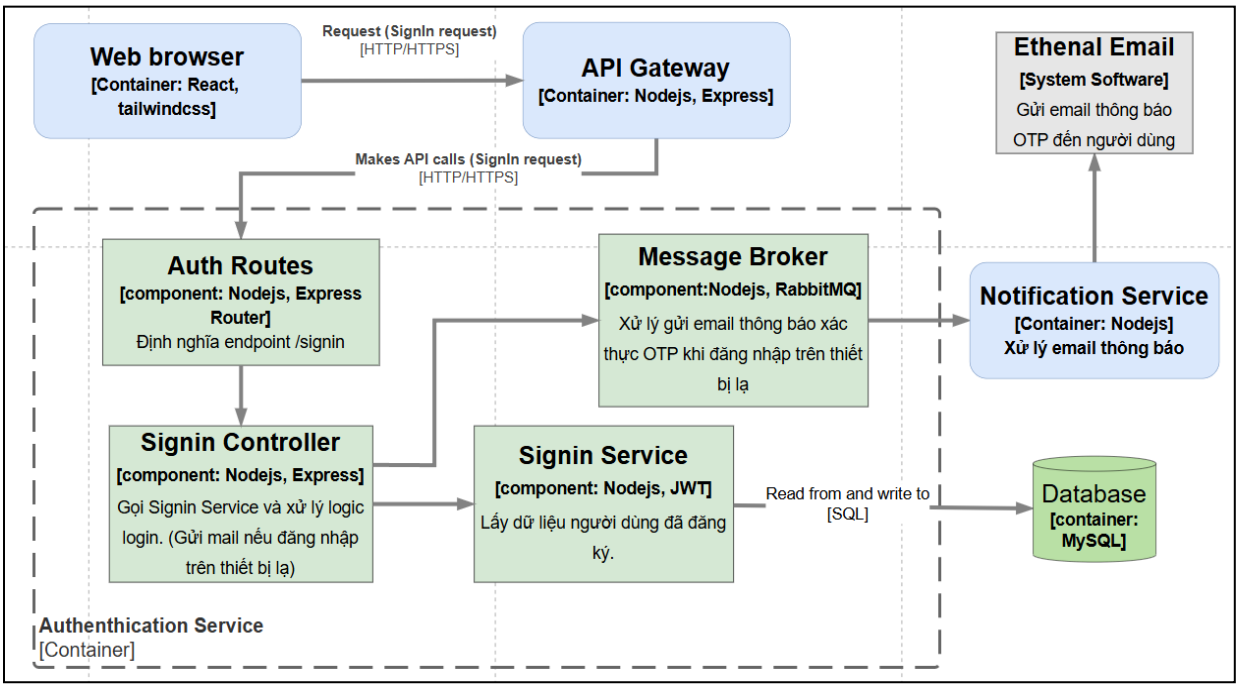
- **Giải thích sơ đồ:**
  - User: là người dùng, sẽ tương tác với hệ thống.
  - Container: Là một phần của hệ thống chạy độc lập, thực hiện một vai trò cụ thể như frontend, backend, database...
  - HTTP/HTTPS: Là giao thức truyền tải dữ liệu giữa client và server (Giao tiếp không mã hóa/Giao tiếp mã hóa).
  - SQL/TCP: Ngôn ngữ dùng để truy vấn và thao tác với cơ sở dữ liệu quan hệ. Và giao thức truyền thông tin giữa các máy tính qua mạng.
- **Giải thích cách hoạt động – Sign In Feature:**
  - Người dụng nhập thông tin vào form đăng nhập rồi bấm nút “Sign In”.
  - Lúc này, request “Sign In” được gửi từ Web Browser đến API Gateway bằng phương thức HTTP/HTTPS.
  - API Gateway sẽ kiểm tra và xác đỉnh request nên được gửi đến Authenthication Service.
  - Authenthication Service sẽ xử lý logic của chức năng Sign In và xảy ra 2 luồng:
    - Luồng 1: Xử lý logic của chức năng Sign In với database.

- Luồng 2: Xử lý logic gửi mail của chức năng Sign In với Notification Services. => Gửi mail đến cho người dùng.

1.3, Component Diagram

Component Diagram là sơ đồ mô tả chi tiết hệ thống bên trong các container như thế nào. Để giúp các developer hiểu rõ kiến trúc bên trong một module hay dịch vụ.

- Hiển thị cấu trúc nội bộ và cách các component tương tác với nhau.
- Các component thường đại diện cho một nhóm chức năng cụ thể.
- Giúp lập trình viên hiểu cách phân chia trách nhiệm bên trong ứng dụng.



Hình 4.1.3 – Component Diagram (Ex: Chức năng đăng nhập)

- **Giải thích sơ đồ:**
  - Component: Là các phần logic nhỏ hơn nằm bên trong một Container, như các module, class, service, controller...
  - Container: Là một dịch vụ riêng biệt trong hệ thống.
  - Software System: Là một hệ thống thực hiện một chức năng nhất định cho người dùng hoặc các hệ thống khác.
- **Giải thích cách hoạt động – Sign In Feature:**
  - Người dụng nhập thông tin vào form đăng nhập rồi bấm nút “Sign In”.

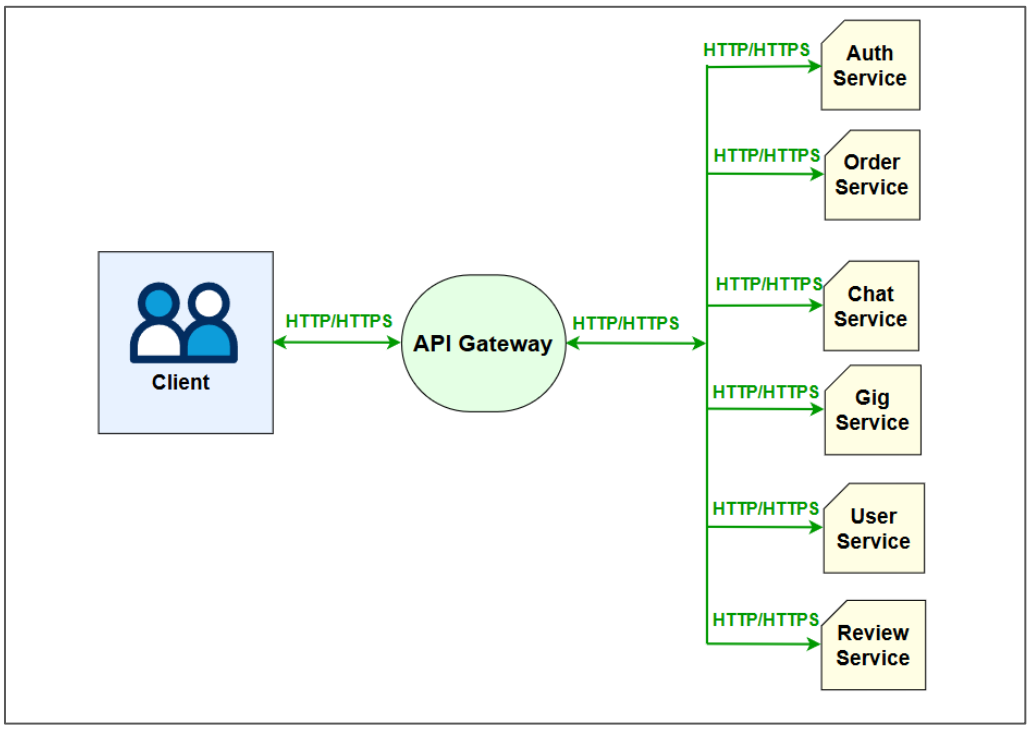
- Lúc này, request “Sign In” được gửi từ Web Browser đến API Gateway bằng phương thức HTTP/HTTPS.
- API Gateway sẽ kiểm tra và xác đỉnh request nên được gửi đến Authenthication Service.
- Request đi vào Componet “Auth Route” của Auth Services. Nhiệm vụ của “Auth Route” sẽ định nghĩa endpoint của “/signin”, endpoint “/signin” là một URL cụ thể tại một HTTP method mà Auth Service cung cấp. Tiếp tục request được định tuyến đến “Auth Controller”
- “Auth Controller” có 2 luồng đi:
  - Luồng thứ 1: Xử lý logic request Sign In bằng cách gửi request đến “Auth Services”, “Auth Services” có nhiệm vụ lấy dữ liệu của người dùng đã đăng ký trong DB, để “Auth Controller” xử lý logic.
  - Luồng thứ 2: Trong trường hợp người dùng đăng nhập trên một thiết bị khác hoặc trình duyệt khác. “Auth Controller” sẽ kiểm tra và gửi một event (dạng message) đến RabbitMQ (Message Broker). RabbitMQ sẽ queue event này đến Notification Service. Notification Service sẽ nhận được event và thực hiện gửi email chứa mã OTP xác thực đến người dùng.

2, Giao tiếp trong ShopMicro

2.1, Giao tiếp giữa Client & Services

API Gateway đóng vai trò là điểm vào trung tâm, quản lý các tương tác giữa Client và các Microservices. Các vai trò chính bao gồm:

- **Chức năng**: Điều hướng các request từ Client đến các Microservices phù hợp.
- **Bảo mật**: Thực hiện kiểm tra xác thực và phân quyền.
- **Công nghệ sử dụng**: Được phát triển bằng Node.js và Express, tận dụng khả năng xử lý yêu cầu HTTP/HTTPS nhanh chóng và linh hoạt.



Hình 4.2.1: Giao tiếp giữa Client & Services

2.2, RESTful API và Vai trò trong API Gateway

RESTful API (Representational State Transfer API) là một phong cách kiến trúc cho các ứng dụng web, dựa trên các nguyên tắc của REST. Nó sử dụng giao thức HTTP/HTTPS để cho phép các hệ thống khác nhau (Client và Server) giao tiếp thông qua các yêu cầu và phản hồi tiêu chuẩn. RESTful API hoạt động dựa trên các đặc điểm chính như:

- **Stateless**: Mỗi yêu cầu từ Client chứa toàn bộ thông tin cần thiết, Server không lưu trữ trạng thái giữa các yêu cầu.
- **Resources**: Dữ liệu được biểu diễn dưới dạng tài nguyên, được truy cập qua URL.
- **Phương thức HTTP**: Sử dụng các phương thức tiêu chuẩn như GET, POST, PUT, DELETE để thực hiện các thao tác CRUD (Create, Read, Update, Delete).
- **Định dạng dữ liệu**: Hỗ trợ các định dạng như JSON hoặc XML để trao đổi thông tin.

Trong ShopMicro, RESTful API được triển khai để kết nối Client với các Microservices (Auth Service, Order Service, v.v.) qua HTTP/HTTPS, đảm bảo tính bảo mật và chuẩn hóa. RESTful API gồm có vai trò như:

- **Khả năng mở rộng**: Với kiến trúc không trạng thái, hệ thống dễ dàng mở rộng bằng cách thêm các server hoặc Microservices mà không ảnh hưởng đến hiệu suất.

- **Tính linh hoạt**: RESTful API hỗ trợ nhiều loại Client (Web, Mobile) và dễ tích hợp với các công nghệ khác nhau.
- **Hiệu suất cao**: Sử dụng caching (lưu trữ tạm) để giảm tải cho Server, đặc biệt quan trọng trong Thương mại điện tử với lượng truy cập lớn.
- **Bảo mật**: Sử dụng HTTPS để mã hóa dữ liệu, bảo vệ thông tin nhạy cảm như thông tin thanh toán hoặc tài khoản người dùng.

3, Khung nhìn mức dữ liệu

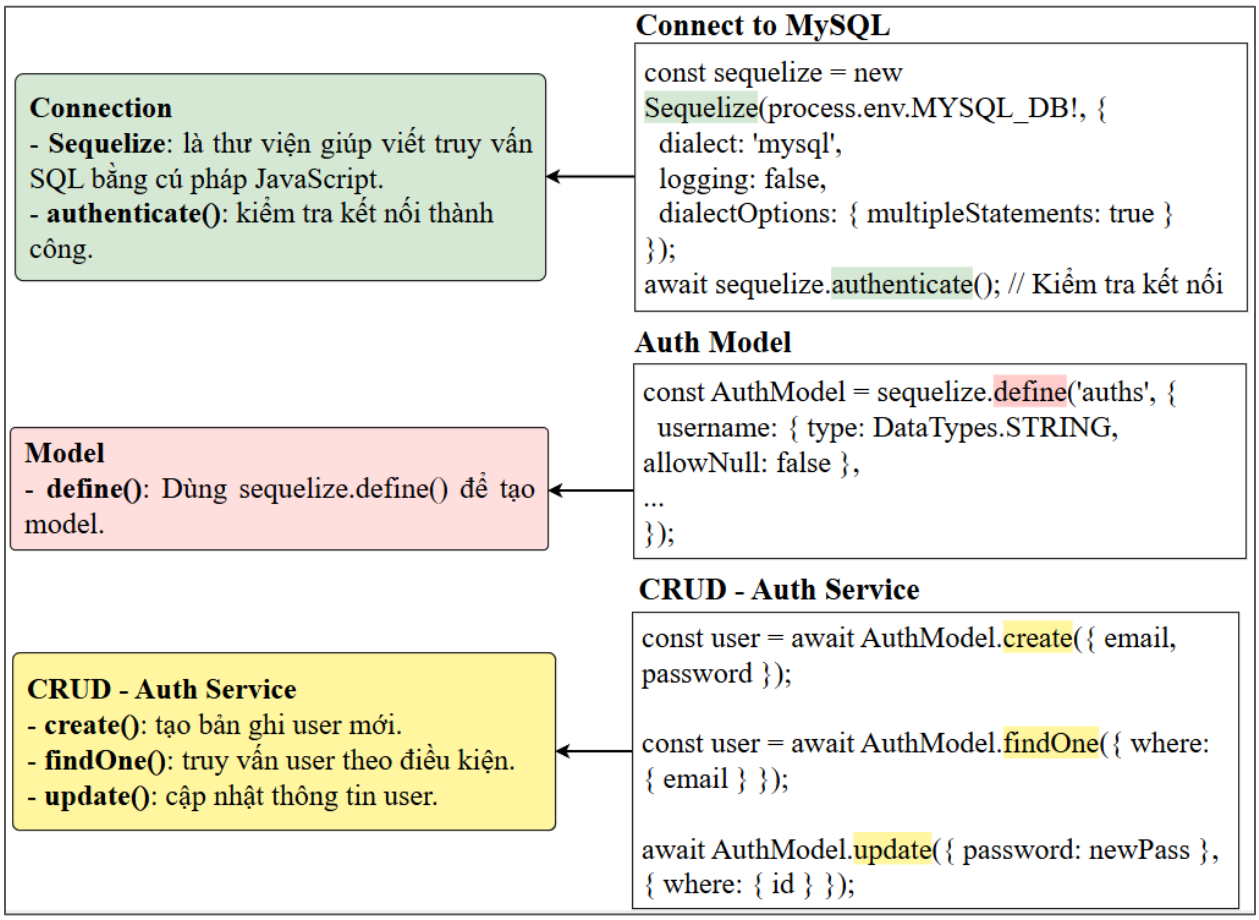
Trong hệ thống Ecommerce được xây dựng theo kiến trúc microservices, mỗi service sử dụng một loại cơ sở dữ liệu khác nhau (MySQL, MongoDB, PostgreSQL). Mỗi loại cơ sở dữ liệu được lập trình tương tác theo cách riêng, dựa trên công cụ hoặc thư viện phù hợp. Có ba cách tiếp cận chính được sử dụng trong hệ thống:

- **Object-Relational Mapping (ORM)**
    - ORM là kỹ thuật ánh xạ giữa các đối tượng trong chương trình (object in code) và các bảng trong cơ sở dữ liệu quan hệ.
    - Khi sử dụng ORM, lập trình viên có thể thao tác với dữ liệu bằng các đối tượng thay vì viết truy vấn SQL trực tiếp.
- **Object-Document Mapping (ODM)**
    - DM tương tự ORM nhưng dành cho cơ sở dữ liệu **phi quan hệ** (NoSQL) như MongoDB.
    - Nó cho phép ánh xạ giữa đối tượng trong code và tài liệu (document) trong MongoDB.
- **Truy vấn thủ công (Raw Query)**
    - Với một số dịch vụ như PostgreSQL trong hệ thống, thay vì sử dụng ORM/ODM, hệ thống sử dụng **thư viện truy vấn gốc** để thao tác với cơ sở dữ liệu.

3.1, MySQL (ORM - Sequelize)

- Auth Service sử dụng MySQL làm cơ sở dữ liệu chính và thư viện Sequelize để thực hiện ORM.
- Mỗi bảng dữ liệu được ánh xạ thành một **model** trong ứng dụng

- Các thao tác như tạo người dùng (create), tìm người dùng (findOne), cập nhật dữ liệu,... được thực hiện thông qua các phương thức của Sequelize thay vì viết SQL thủ công.

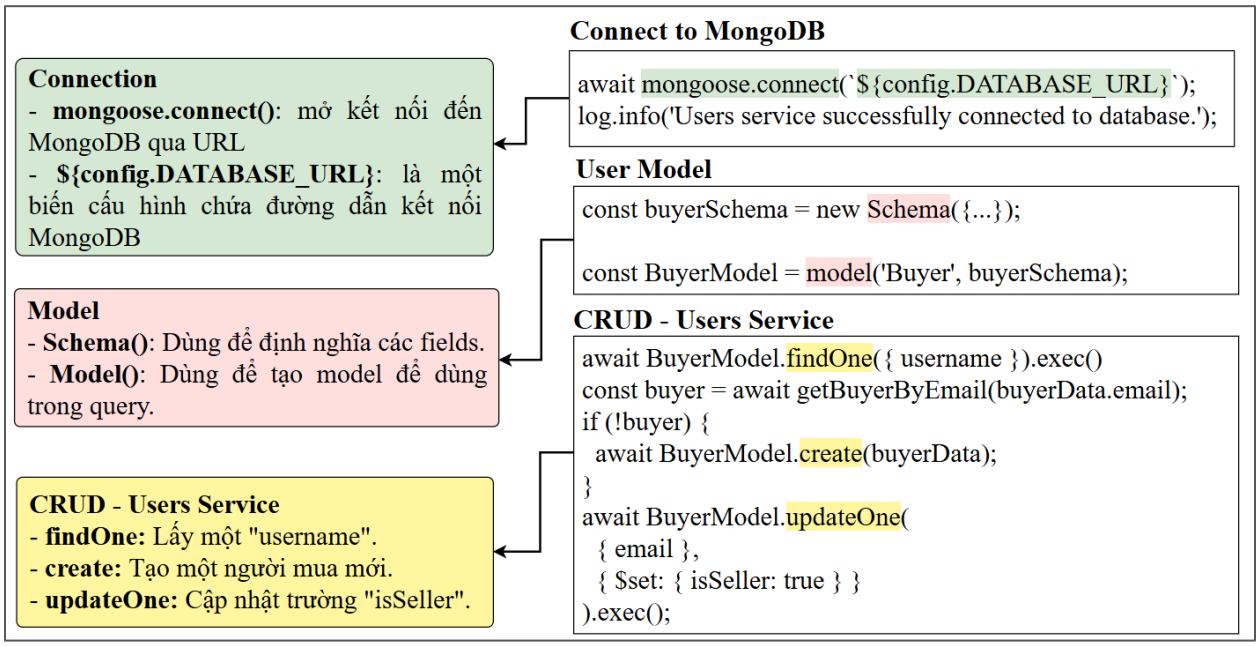


Hình 4.3.1: MySQL - ORM

3.2, MongoDB (ODM - Mongoose)

- Các services như User, Gig, Chat, Order đều sử dụng MongoDB – một cơ sở dữ liệu NoSQL dạng document.
- Hệ thống sử dụng thư viện Mongoose, một ODM cho MongoDB, để định nghĩa schema và ánh xạ chúng thành model.
- Các thao tác như tạo người dùng (create), tìm kiếm (findOne), cập nhật (updateOne) đều được thực hiện bằng cú pháp của Mongoose, phù hợp với lập trình hướng đối tượng.
    - Mongoose hỗ trợ:
    - Kiểm tra kiểu dữ liệu (type validation)

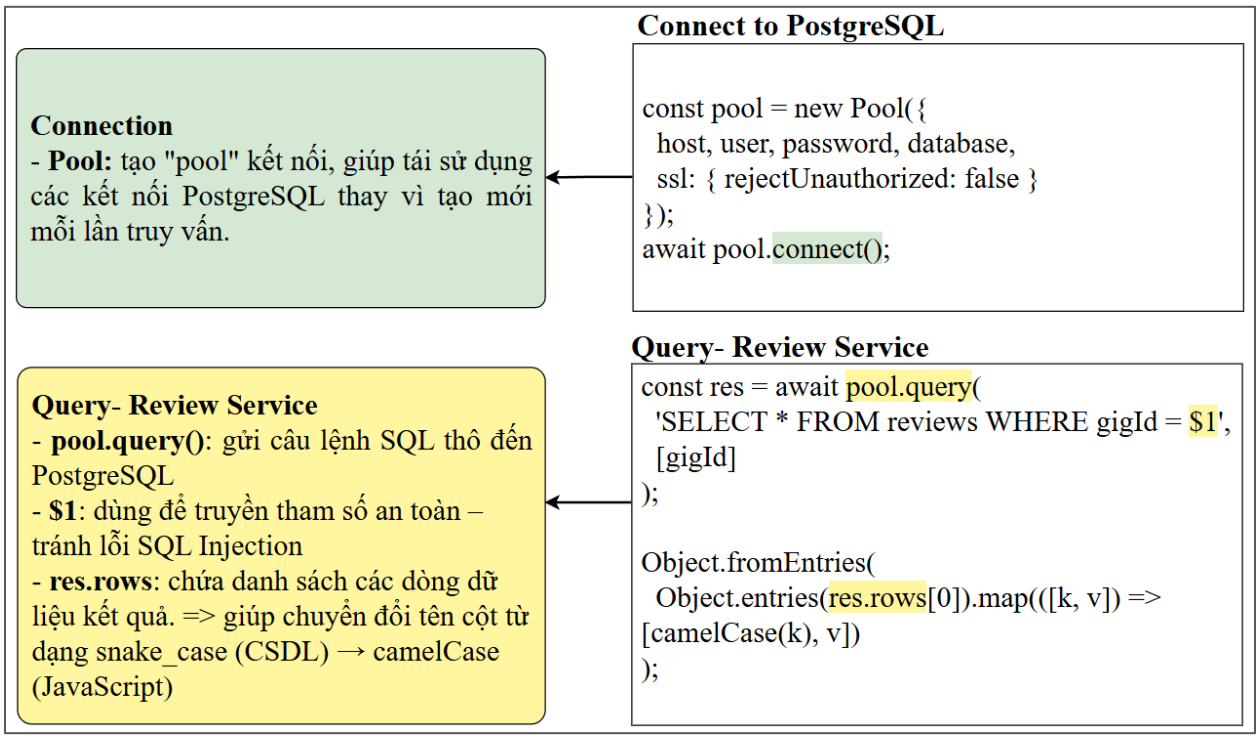
- Indexing
- Liên kết dữ liệu giữa các document (population)
- Middleware (hooks)



Hình 4.3.2: MongoDB – ODM

3.3, PostgreSQL (Raw Query)

- Review Service sử dụng PostgreSQL làm cơ sở dữ liệu chính.
- Hệ thống không sử dụng ORM mà sử dụng thư viện pg của Node.js, cho phép gửi truy vấn SQL trực tiếp đến cơ sở dữ liệu.
- Các thao tác như INSERT, SELECT, UPDATE đều được viết bằng raw SQL string.
- Dữ liệu trả về từ truy vấn (dạng rows) được ánh xạ thủ công sang object ứng dụng bằng cách duyệt và đổi key theo định dạng cần thiết.
- Mặc dù cách này cung cấp sự linh hoạt và kiểm soát cao, nhưng đòi hỏi lập trình viên phải xử lý thủ công nhiều khâu như:
  - Viết và bảo trì truy vấn SQL
  - Quản lý ánh xạ dữ liệu trả về
  - Đảm bảo an toàn truy vấn (prepared statements)



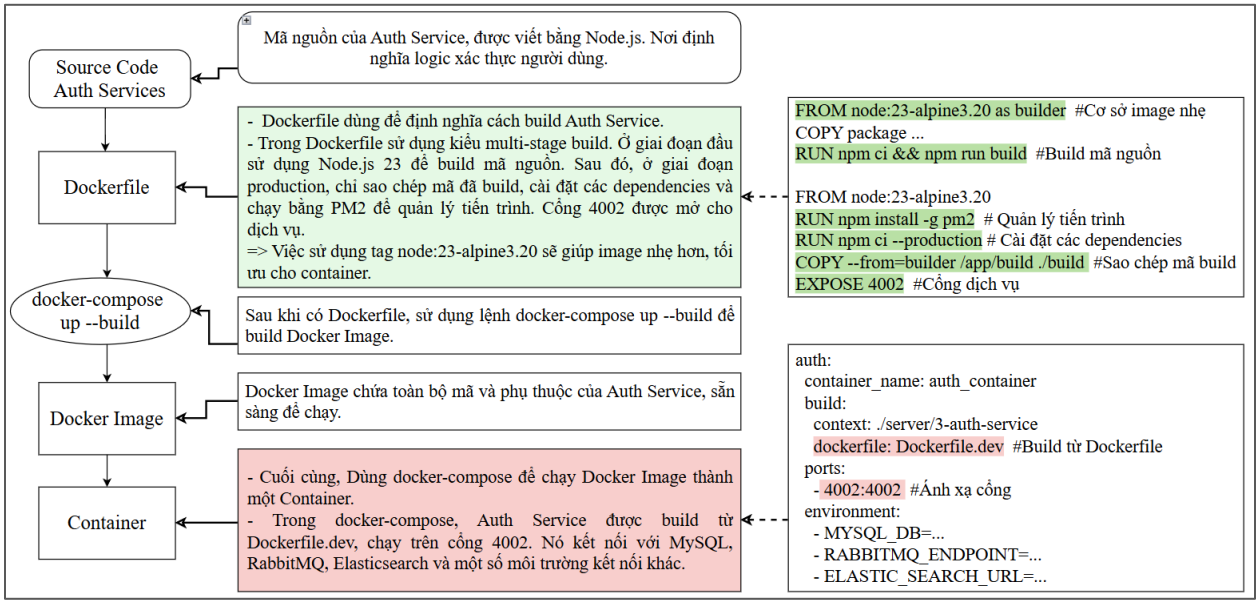
Hình 4.3.3: PostgreSQL – Raw Query

4, Triển khai hệ thống

Docker là một nền tảng mã nguồn mở giúp đóng gói ứng dụng và các thành phần phụ thuộc của nó vào trong một đơn vị độc lập gọi là container. Điều này giúp ứng dụng có thể chạy ổn định trên nhiều môi trường khác nhau, từ máy phát triển cá nhân đến server production.

Ưu điểm khi sử dụng Docker trong triển khai:

- Đóng gói toàn bộ ứng dụng: Docker Image chứa đầy đủ mã nguồn, thư viện và môi trường chạy ứng dụng.
- Tính nhất quán cao: Ứng dụng chạy giống nhau ở mọi nơi, giảm lỗi do khác biệt môi trường.
- Triển khai nhanh chóng: Tạo và chạy container chỉ trong vài giây.
- Tích hợp CI/CD: Tự động build, test và deploy dễ dàng trong pipeline.



Hình 4.4(1): Quy trình triển khai hệ thống

Quy trình triển triển khai trong ShopMicro sử dụng Docker bao gồm các bước chính sau:

Bước 1: Mã nguồn

- Các dịch vụ được viết bằng Node.js.
- Chứa toàn bộ logic, thư viện và các thành phần phụ thuộc cần thiết.

Bước 2: Dockerfile

- Dockerfile sẽ định nghĩa cách build Service.
- Trong mỗi service thì Docker sử dụng multi-stage build để tối ưu hóa kích thước image.
- Giai đoạn đầu: sử dụng node:23-alpine3.20 as builder để:
    - Cài đặt dependencies (npm ci)
    - Build mã nguồn (npm run build)
- Giai đoạn thứ hai:
    - Sử dụng lại image node:23-alpine3.20 nhẹ và tối ưu hơn.
    - Cài đặt PM2 để quản lý tiến trình (npm install -g pm2)
    - Cài đặt dependencies production (npm ci –production)
    - Sao chép mã đã build từ stage trước.
    - Mở cổng dịch vụ để expose.

=> Việc dùng alpine3.20 giúp image nhẹ và tối ưu hơn cho môi trường production.

```
FROM node:23-alpine3.20 as builder

WORKDIR /app
COPY package*.json ./
COPY tsconfig.json ./
COPY .npmrc ./
COPY src ./src
RUN npm install -g npm@latest
RUN npm ci && npm run build

FROM node:23-alpine3.20

WORKDIR /app
RUN apk add --no-cache curl
COPY package*.json ./
COPY tsconfig.json ./
COPY .npmrc ./
RUN npm install -g pm2 npm@latest
RUN npm ci --production
COPY --from=builder /app/build ./build

EXPOSE 4002

CMD [ "npm", "run", "start" ]
```

Hình 4.4(2): Minh họa Dockerfile

Bước 3: Command

- Sau khi đã có Dockerfile thì sử dụng lệnh “docker-compose up –build” để Build Image cho các services. Docker Image tạo ra sẽ chứa toàn bộ mã nguồn và dependencies của Auth Service, sẵn sàng để chạy.

Bước 4: Container

- Sau khi đã build xong Image, dùng docker-compose để khởi chạy container, tạo container từ image của Auth Service.

```yaml
auth:
  container_name: auth_container
  build:
    context: ./server/3-auth-service
    dockerfile: Dockerfile.dev
  restart: always
  ports:
    - 4002:4002
  env_file: ./server/3-auth-service/.env
  environment:
    - ENABLE_APM=1
    - GATEWAY_JWT_TOKEN=32ab09f792afefd1ef9655e60d44a333
    - JWT_TOKEN=f2ce50a9a4fe338cd09ed21acc4b82d1
    - NODE_ENV=development
    - AP_GATEWAY_URL=http://gateway_container:4000
    - CLIENT_URL=http://localhost:3000
    - RABBITMQ_ENDPOINT=amqp://jobber:jobberpass@rabbitmq_container:5672
    - MYSQL_DB=mysql://user:api@mysql_container:3306/jobber_auth
    - CLOUD_NAME=dwkc1k4et
    - CLOUD_API_KEY=845112264248622
    - CLOUD_API_SECRET=Crc0hIoofQuB2lEz4YnO9nv5RVk
    - ELASTIC_SEARCH_URL=http://elastic:admin1234@elasticsearch_container:9200
    - ELASTIC_APM_SERVER_URL=http://apm_server_container:8200
    - ELASTIC_APM_SECRET_TOKEN=
  depends_on:
    - elasticsearch
    - mysql
```

Hình 4.4(3): Minh họa docker-compose.yaml

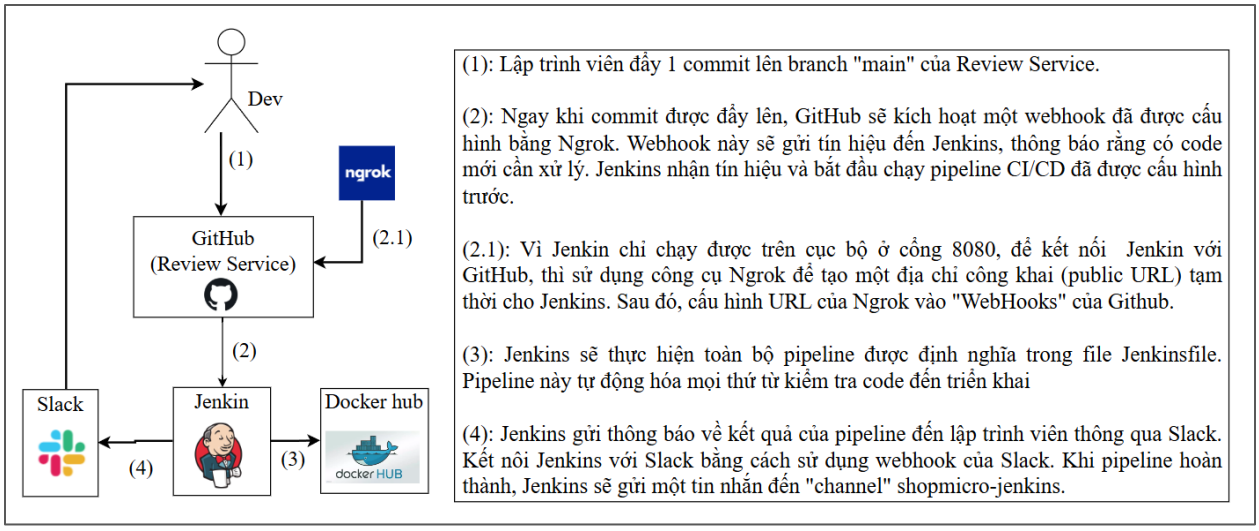
5, CI/CD với Jenkins

Jenkins là một công cụ mã nguồn mở, được sử dụng rộng rãi để tự động hóa các quy trình phát triển phần mềm, đặc biệt trong việc tích hợp liên tục (Continuous Integration - CI) và triển khai liên tục (Continuous Deployment - CD). Jenkins cho phép lập trình viên thiết lập các pipeline (luồng công việc) để tự động hóa các bước như kiểm tra code, build, test, và triển khai ứng dụng.

- Ưu điểm của Jenkins:
    - Jenkins hỗ trợ nhiều plugin (hiện có hơn 1.800 plugin), cho phép tích hợp với hầu hết các công cụ như Git, Docker, Slack, và nhiều công nghệ khác.
    - Giúp tự động hóa toàn bộ quy trình phát triển phần mềm, giảm thiểu lỗi do con người và tiết kiệm thời gian.
    - Hỗ trợ kiến trúc phân tán (Master-Agent), cho phép chạy nhiều tác vụ đồng thời trên các máy khác nhau.
    - Jenkins cho phép viết pipeline bằng code (Jenkinsfile), giúp quản lý và tái sử dụng dễ dàng.

5.1, Luồng hoạt động CI/CD trong hệ thống

Hệ thống CI/CD của ShopMicro sử dụng Jenkins để tự động hóa quy trình phát triển và triển khai cho các services. Quy trình này bao gồm 4 bước chính, được minh họa trong sơ đồ dưới đây:



Hình 4.5.1: Luồng hoạt động của CI/CD

1. Lập trình viên đẩy commit lên GitHub: Lập trình viên đẩy một commit lên nhánh main của Service trên GitHub.
2. Webhook kích hoạt Jenkins: GitHub gửi tín hiệu qua webhook (được cấu hình với Ngrok để tạo URL công khai) đến Jenkins, thông báo có code mới. Jenkins nhận tín hiệu và chạy pipeline CI/CD.
3. Jenkins thực thi pipeline CI/CD: Pipeline được định nghĩa trong Jenkinsfile, bao gồm các bước như tải code, kiểm tra chất lượng (lint, format, unit test), build image Docker, và đẩy lên Docker Hub.
4. Thông báo kết quả qua Slack: Sau khi pipeline hoàn thành (thành công hoặc thất bại), Jenkins gửi thông báo đến lập trình viên qua kênh Slack shopmicro-jenkins, bao gồm chi tiết về trạng thái build, tác giả commit, và thời gian thực hiện.

5.2, Quá trình cài đặt và triển khai CI/CD.

- **Khởi tạo Jenkins và Agent**
    - Hệ thống sử dụng docker-compose.yaml để build hai container:
        - jenkins_container: Jenkins master, chạy trên cổng 8080, sử dụng image jenkins/jenkins:lts.

- jenkins_agent_container: Jenkins agent, sử dụng image jenkins/ssh-agent:jdk17, hỗ trợ SSH để kết nối với master.

```
jenkins:
  container_name: jenkins_container
  image: jenkins/jenkins:lts
  privileged: true
  user: root
  ports:
    - 8080:8080
    - 50000:50000
  volumes:
    - ./docker-volumes/jenkins_compose/jenkins_configuration:/var/jenkins_home
    - /var/run/docker.sock:/var/run/docker.sock

jenkins-agent:
  container_name: jenkins_agent_container
  image: jenkins/ssh-agent:jdk17
  privileged: true
  user: root
  expose:
    - 22
  environment:
    - JENKINS_AGENT_SSH_PUBKEY=ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAABgQDF3CYLI/zF
```

4.5.2(1): Minh họa cách build Jenkin

  - Sau khi build thành công trên Docker Desktop, vào jenkins_agent_container qua mục "exec" và chạy các lệnh:
    - apt install và apt install docker.io: Cài Docker để agent có thể build image.
    - docker service start: Khởi động Docker (lưu ý: bước này có thể không cần nếu không dùng Docker Swarm).
  - Việc này đảm bảo agent có thể truy cập Docker socket để thực hiện các tác vụ liên quan đến Docker.

- **Cấu hình Jenkins**
  - Truy cập Jenkins qua http://localhost:8080 và đăng nhập.
  - Trong mục "Manage", có cấu hình:
    - Nodes: Thêm jenkins_agent_container làm node với nhãn Jenkins-Agent.
    - Credentials: Thêm credentials cho GitHub (github) và Docker Hub (dockerhub).

        - Plugins: Cài các plugin cần thiết như Git, Pipeline, Docker Pipeline, NodeJS, và Slack Notification.
    - Sau khi cấu hình thành công, tạo một pipeline mới cho các Services bằng cách chọn "New Item", chọn loại "Pipeline".

- **Tạo và sử dụng functions.groovy**
    - Tạo một file functions.groovy chứa các hàm Groovy hỗ trợ pipeline, chẳng hạn:
        - readCommitAuthor: Đọc thông tin tác giả commit từ Git.
        - durationTime: Tính thời gian build.
        - notifySlack: Gửi thông báo đến Slack.
    - File này được đẩy lên repository GitHub có tên là jenkins-automation. Trong pipeline, Jenkins sẽ tải file này để sử dụng các hàm hỗ trợ.

```groovy
def readCommitAuthor() {
    sh '''#!/bin/bash
        git rev-parse HEAD | tr '\n' ' ' > gitCommit
        git show --format="%aN <%aE>" ${gitCommit} | head -1 | tr '\n' ' ' > gitCommitAuthor
    '''
    return readFile('gitCommitAuthor')
}

def durationTime(m1, m2) {
    int timecase = m2 - m1

    int seconds = (int) (timecase / 1000)
    int minutes = (int) (timecase / (60*1000))
    int hours = (int) (timecase / (1000*60*60))

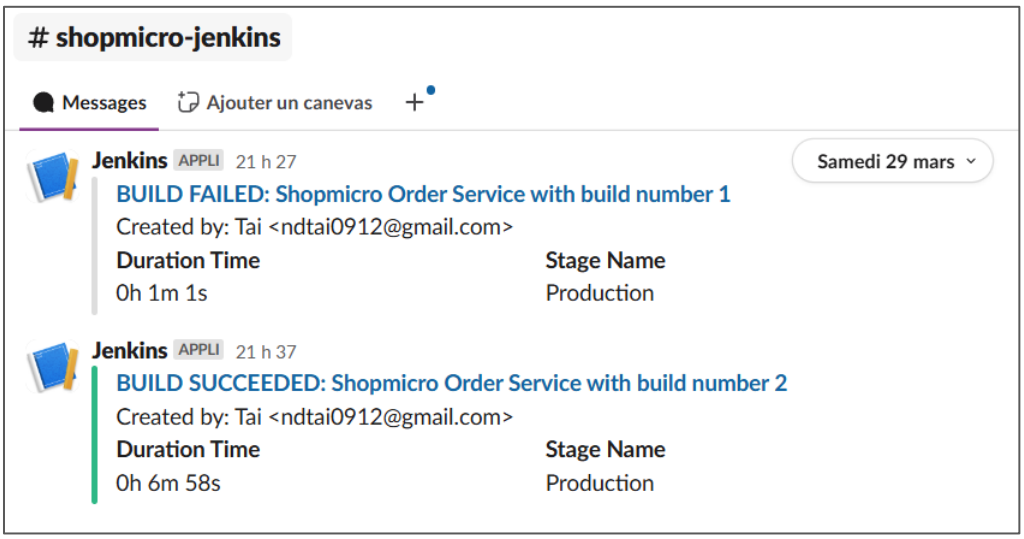
    return hours.mod(24) + "h " + minutes.mod(60) + "m " + seconds.mod(60) + "s"
}

def notifySlack(text, channel, attachments) {
    def slackURL = 'https://hooks.slack.com/services/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'
    def jenkinsIcon = 'https://a.slack-edge.com/205a/img/services/jenkins-ci_72.png'

    def payload = JsonOutput.toJson([
        text: text,
        channel: channel,
        username: "jenkins",
        icon_url: jenkinsIcon,
        attachments: attachments
    ])
```

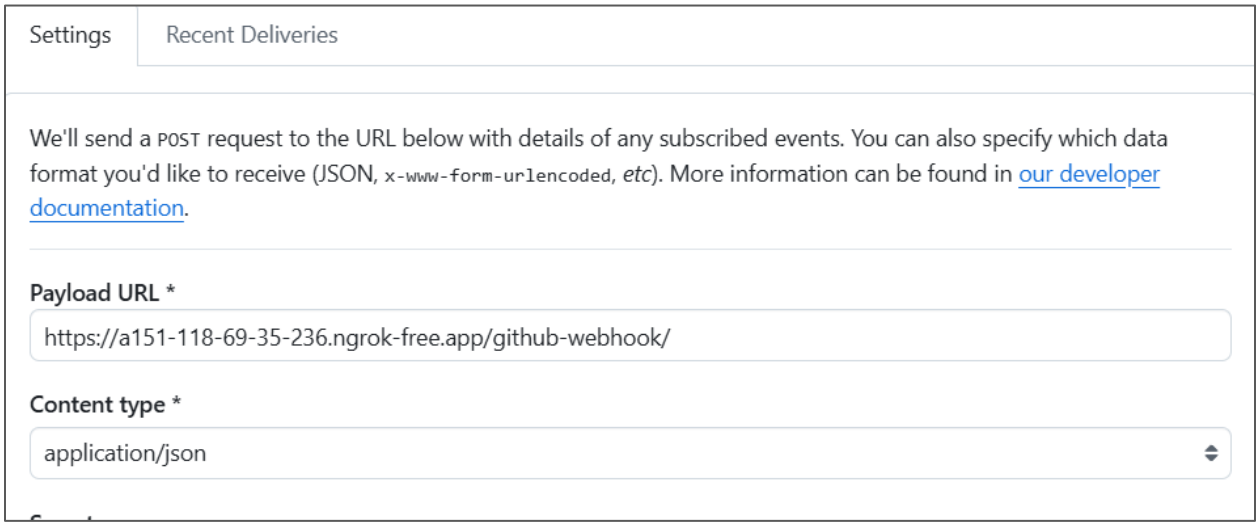
4.5.2(2): Minh họa các hàm hỗ trợ Pipeline

- **Tích hợp với Slack**
    - Tải ứng dụng Slack và tạo webhook qua api.slack.com.
    - Sau khi kết nối thành công, Jenkins sử dụng webhook này (được lưu trong functions.groovy) để gửi thông báo đến kênh shopmicro-jenkins.



4.5.2(3): Minh họa thông báo cho Pipeline

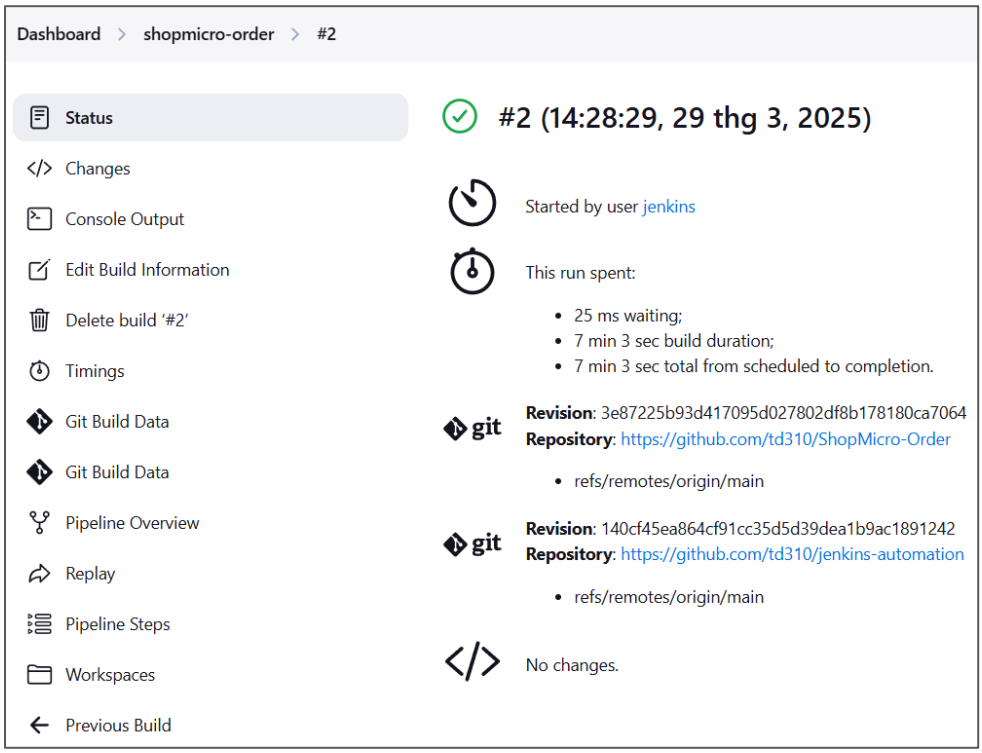
- Tích hợp với GitHub qua Webhook
    - Để Jenkins nhận tín hiệu từ GitHub, tải công cụ Ngrok và tạo URL công khai: https://a151-118-69-35-236.ngrok-free.app/github-webhook/.
    - Trong mục "Webhooks" của Service trên GitHub, có cấu hình:
        - Payload URL: URL Ngrok ở trên.
        - Content type: application/json.
        - Events: Chọn push để kích hoạt pipeline khi có commit mới.



4.5.2(4): Minh họa Webhook Github

- **Kiểm tra CI/CD**
    - Sau khi hoàn tất thiết lập, đẩy một commit mới lên nhánh main của Review Service.

- Jenkins tự động nhận tín hiệu qua webhook, chạy pipeline, và thực hiện các bước: kiểm tra code, build image, đẩy lên Docker Hub, và gửi thông báo qua Slack.



4.5.2(5): Minh họa build Pipeline thành công

6, Resilience: Giám sát hệ thống

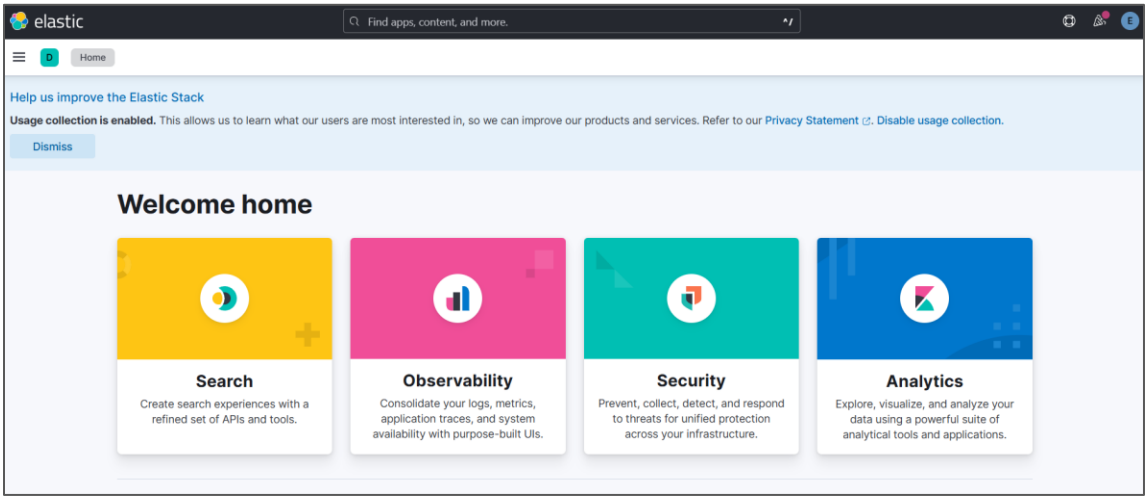
Để đảm bảo hệ thống của chúng ta hoạt động ổn định và có khả năng phục hồi (resilience), chúng ta sử dụng một bộ công cụ mạnh mẽ để theo dõi, phân tích, và phát hiện các vấn đề kịp thời. Dưới đây là các thành phần chính trong hệ thống giám sát:

6.1, Elasticsearch

- Elasticsearch là một công cụ tìm kiếm và phân tích dữ liệu thời gian thực, hoạt động như một "ngân hàng lưu trữ thông minh". Nó được thiết kế để thu thập, lưu trữ và xử lý nhanh chóng các loại dữ liệu lớn như log (nhật ký hoạt động), metric (chỉ số hiệu suất), trace (dấu vết giao dịch), và nhiều dữ liệu khác từ hệ thống.

6.2, Kibana

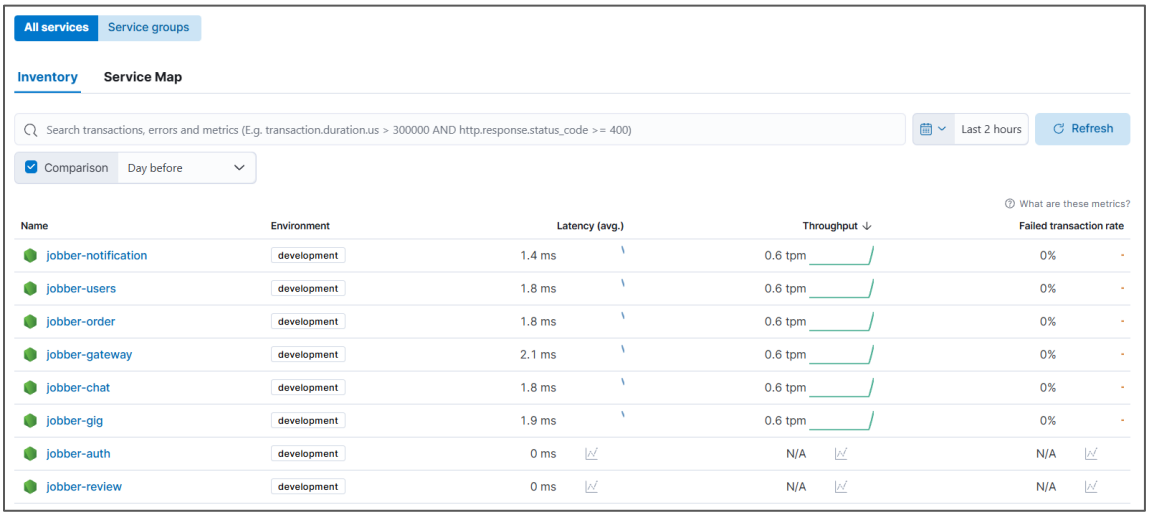
- Kibana là giao diện web thân thiện, hoạt động như "cửa sổ" để trực quan hóa dữ liệu được lưu trong Elasticsearch. Thay vì phải đọc các dòng log thô hoặc số liệu phức tạp, Kibana biến chúng thành biểu đồ, bảng biểu, hoặc bản đồ dễ nhìn.



Hình 4.6.2: Giao diện Kibana (Elasticsearch)

6.3, APM Server (Application Performance Monitoring)

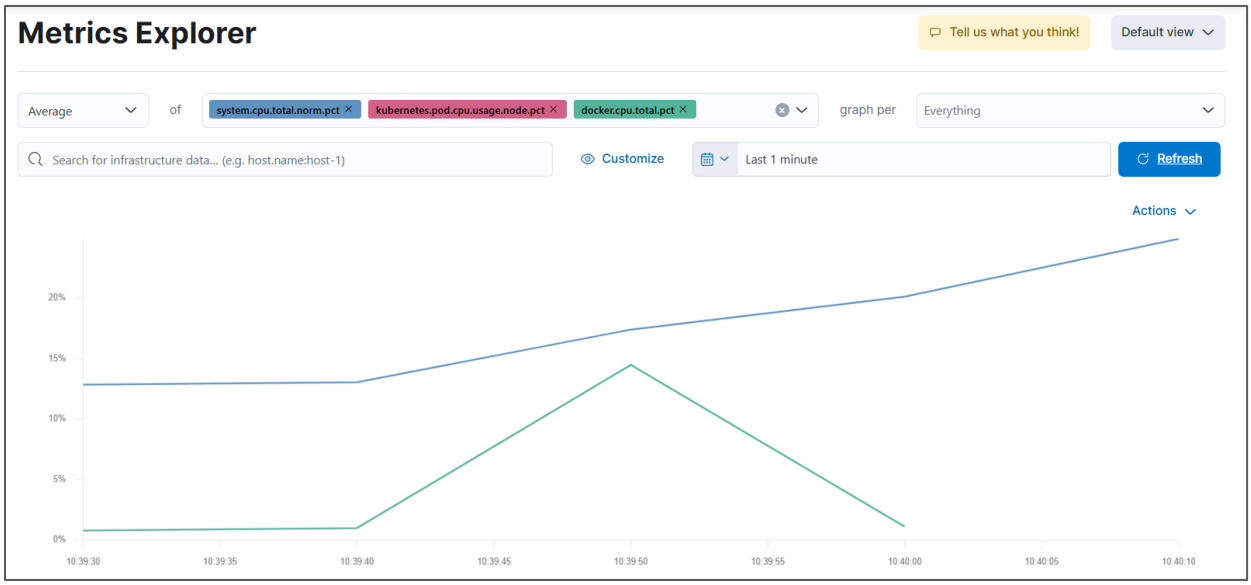
- APM Server là "người trung gian" trong hệ thống giám sát, đóng vai trò thu thập dữ liệu hiệu suất từ các ứng dụng. Nó làm việc với các agent – những đoạn mã nhỏ được cài đặt trực tiếp bên trong từng dịch vụ (service) trong hệ thống của chúng ta.
- Công dụng thực tế: APM giúp theo dõi chi tiết hiệu suất của ứng dụng, như:
    - Xem một yêu cầu từ người dùng mất bao lâu để xử lý.
    - Kiểm tra tốc độ và hiệu quả khi ứng dụng giao tiếp với cơ sở dữ liệu.
    - Quan sát các bước xử lý bên trong ứng dụng.
    - Phát hiện các lỗi cụ thể (ví dụ: lỗi 500) và nguyên nhân gây ra chúng.



Hình 4.6.3: APM Server

6.4, Metricbeat

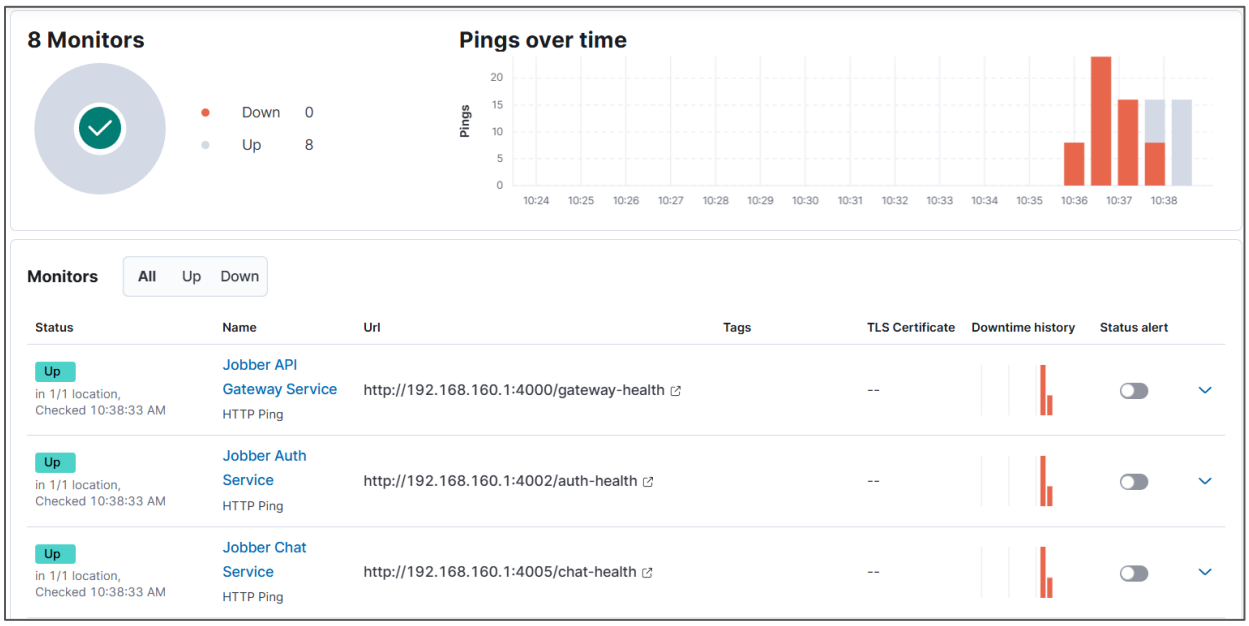
- Metricbeat là một "điệp viên nhẹ cân" được cài đặt trực tiếp trên các máy chủ hoặc môi trường triển khai. Nó có nhiệm vụ thu thập các chỉ số (metrics) liên quan đến hệ điều hành (như CPU, RAM) và môi trường chạy ứng dụng (như Docker).
- Công dụng thực tế: Metricbeat giúp giám sát mức sử dụng tài nguyên của hệ thống. Nó có thể:
    - Phát hiện tình trạng quá tải (ví dụ: CPU đạt 90% trong thời gian dài).
    - Nhận diện rò rỉ tài nguyên (như bộ nhớ bị tiêu tốn mà không giải phóng).
    - Ví dụ: Nếu máy chủ đột nhiên chậm, Metricbeat sẽ báo cáo rằng RAM đã đầy, giúp đội ngũ hành động kịp thời.



Hình 4.6.4: Metricbeat

6.5, Heartbeat

- Heartbeat là công cụ chuyên dụng để kiểm tra trạng thái "sống" (liveness) của các dịch vụ. Nó hoạt động bằng cách gửi các request HTTP định kỳ đến các địa chỉ dịch vụ đã được cấu hình trước.
- Công dụng thực tế: Heartbeat giúp xác minh xem dịch vụ có đang hoạt động bình thường không. Nếu dịch vụ bị treo (hang) hoặc chết (down), Heartbeat sẽ gửi cảnh báo sớm để đội ngũ can thiệp.



Hình 4.6.5: Heartbeat

V, Kết quả đạt được

Sau quá trình phát triển và triển khai, hệ thống ShopMicro đã đạt được những kết quả đáng kể, đáp ứng các mục tiêu đề ra và mang lại nhiều lợi ích cho người dùng. Dưới đây là các thành tựu chính mà nhóm đã đạt được:

1, Hoàn thiện các tính năng chính của hệ thống

Hệ thống ShopMicro đã triển khai thành công các dịch vụ và tính năng quan trọng, bao gồm:

- Dịch vụ xác thực (Authentication):
  - Đăng ký và đăng nhập an toàn cho người dùng (khách hàng và người bán).
  - Xác thực tài khoản qua email.
  - Quên mật khẩu và đổi mật khẩu an toàn.
- Dịch vụ người dùng (User):
  - Quản lý hồ sơ người dùng (profile).
  - Cho phép người mua trở thành người bán.
  - Quản lý số lượng công việc và đơn hàng.
- Dịch vụ sản phẩm (Gig):
  - Quản lý sản phẩm (thêm, xóa, sửa) cho người bán.
  - Tìm kiếm sản phẩm theo tên, thời gian giao hàng, và khoảng giá.

- Dịch vụ thông báo (Notification):
    - Gửi email thông báo cho các sự kiện quan trọng (xác thực, đơn hàng, trò chuyện).
    - Thông báo real-time cho trò chuyện và đơn hàng.
- Dịch vụ đơn hàng (Order):
    - Đặt đơn hàng và thanh toán an toàn.
    - Quản lý trạng thái đơn hàng (chờ, đang thực hiện, đã giao).
    - Yêu cầu thêm thời gian chuẩn bị đơn hàng.
- Dịch vụ trò chuyện (Chat):
    - Trò chuyện real-time giữa khách hàng và người bán.
    - Gửi "Offer" sản phẩm qua chat.
    - Đánh dấu tin nhắn đã đọc.
- Dịch vụ đánh giá (Review):
    - Đánh giá và xếp hạng sản phẩm sau khi hoàn thành đơn hàng.
    - Phản hồi đánh giá từ người bán.

2, Áp dụng thành công kiến trúc Microservices

- Hệ thống được chia thành 7 dịch vụ độc lập (Authentication, User, Gig, Order, Notification, Chat, Review), mỗi dịch vụ có cơ sở dữ liệu riêng biệt.
- Lợi ích của kiến trúc Microservices:
    - Dễ dàng mở rộng và bảo trì từng dịch vụ mà không ảnh hưởng đến toàn bộ hệ thống.
    - Tăng khả năng chịu tải và độ tin cậy của hệ thống.
    - Cho phép phát triển song song giữa các nhóm làm việc trên các dịch vụ khác nhau.

3, Triển khai hệ thống trên Docker

- Toàn bộ hệ thống được container hóa bằng Docker, với mỗi dịch vụ chạy trong một container riêng.
- Sử dụng Docker Compose để quản lý và triển khai các container trong môi trường phát triển.
- Kết quả:

- Đảm bảo tính nhất quán giữa các môi trường (phát triển, testing, production).
- Triển khai nhanh chóng và dễ dàng trên nhiều nền tảng.
- Giảm thiểu lỗi do khác biệt môi trường.

4, Tích hợp CI/CD với Jenkins

- Thiết lập pipeline CI/CD tự động cho các dịch vụ, bao gồm các bước: kiểm tra code, build image Docker, và đẩy lên Docker Hub.
- Mỗi khi có commit mới trên nhánh main, Jenkins tự động chạy pipeline và thông báo kết quả qua Slack.
- Lợi ích:
  - Tự động hóa quy trình phát triển, giảm thiểu lỗi do con người.
  - Đảm bảo chất lượng code và triển khai nhanh chóng.
  - Tăng tính minh bạch và khả năng theo dõi tiến độ phát triển.

5, Giám sát hệ thống với Elasticsearch, Kibana, APM, Metricbeat, Heartbeat

- Triển khai hệ thống giám sát toàn diện:
  - Elasticsearch và Kibana: Lưu trữ và trực quan hóa log, metric, trace.
  - APM Server: Theo dõi hiệu suất ứng dụng (thời gian phản hồi, lỗi).
  - Metricbeat: Giám sát tài nguyên hệ thống (CPU, RAM, Docker).
  - Heartbeat: Kiểm tra trạng thái "sống" của các dịch vụ.
- Kết quả:
  - Phát hiện và xử lý sự cố nhanh chóng.
  - Tối ưu hóa hiệu suất và tài nguyên hệ thống.
  - Đảm bảo dịch vụ luôn sẵn sàng và hoạt động ổn định.

6, Tối ưu trải nghiệm người dùng với SPA (ReactJS)

- Xây dựng frontend theo mô hình SPA (Single Page Application) với ReactJS, TailwindCSS, và TypeScript.
- Giao diện người dùng mượt mà, tải nhanh, và tương tác tốt trên nhiều thiết bị.
- Tích hợp Socket.IO cho các tính năng real-time như trò chuyện và theo dõi đơn hàng.

7, Đảm bảo bảo mật và hiệu suất

- Sử dụng HTTPS cho toàn bộ giao tiếp giữa client và server.

- Áp dụng caching với Redis để tăng tốc độ truy xuất dữ liệu.
- Bảo mật dữ liệu nhạy cảm (mật khẩu, token) bằng cách mã hóa và quản lý chặt chẽ.

8, Kết luận

Hệ thống ShopMicro đã đạt được các mục tiêu đề ra, bao gồm việc xây dựng một nền tảng freelance hiện đại, dễ mở rộng, và có khả năng phục hồi cao. Các tính năng chính đã được triển khai thành công, mang lại trải nghiệm tốt cho người dùng và đội ngũ phát triển. Việc áp dụng các công nghệ tiên tiến như Microservices, Docker, CI/CD, và hệ thống giám sát đã giúp hệ thống hoạt động ổn định, hiệu quả, và sẵn sàng cho các giai đoạn phát triển tiếp theo.